

Software Quality Assurance and Testing

Instructor: Dr. Nguyễn Quang Vũ

nqvu@vku.udn.vn

Phone: 0901.982.982

COURSE INTRODUCTION

- Lecturer
 - Dr. Nguyen Quang Vu
 - Email: nquu@vku.udn.vn
 - Mobile: (+84) 901.982.982
- Course general information
 - Name of course: Software Quality Assurance Testing
 - Course status for program: Required
 - Knowledge block: Specialized knowledge
 - Number of credits: 3 (60 sessions; 1 session = 50 minutes)

COURSE INTRODUCTION

- Student Task

- Students **should** attend more than 80% of sessions;
- Students are responsible to do all exercises, homework, assignments and major assignment given by instructor in class or at home and submit on time.
- Students can use laptop in class only for learning purpose.
- Regular access to the VKU's eLearning System (<http://elearning.vku.udn.vn/>) for up-to-date course information; for receiving and submitting the exercises, homework, assignments and labs given by instructor.

- Course Plan

- Details in the Course Syllabus;



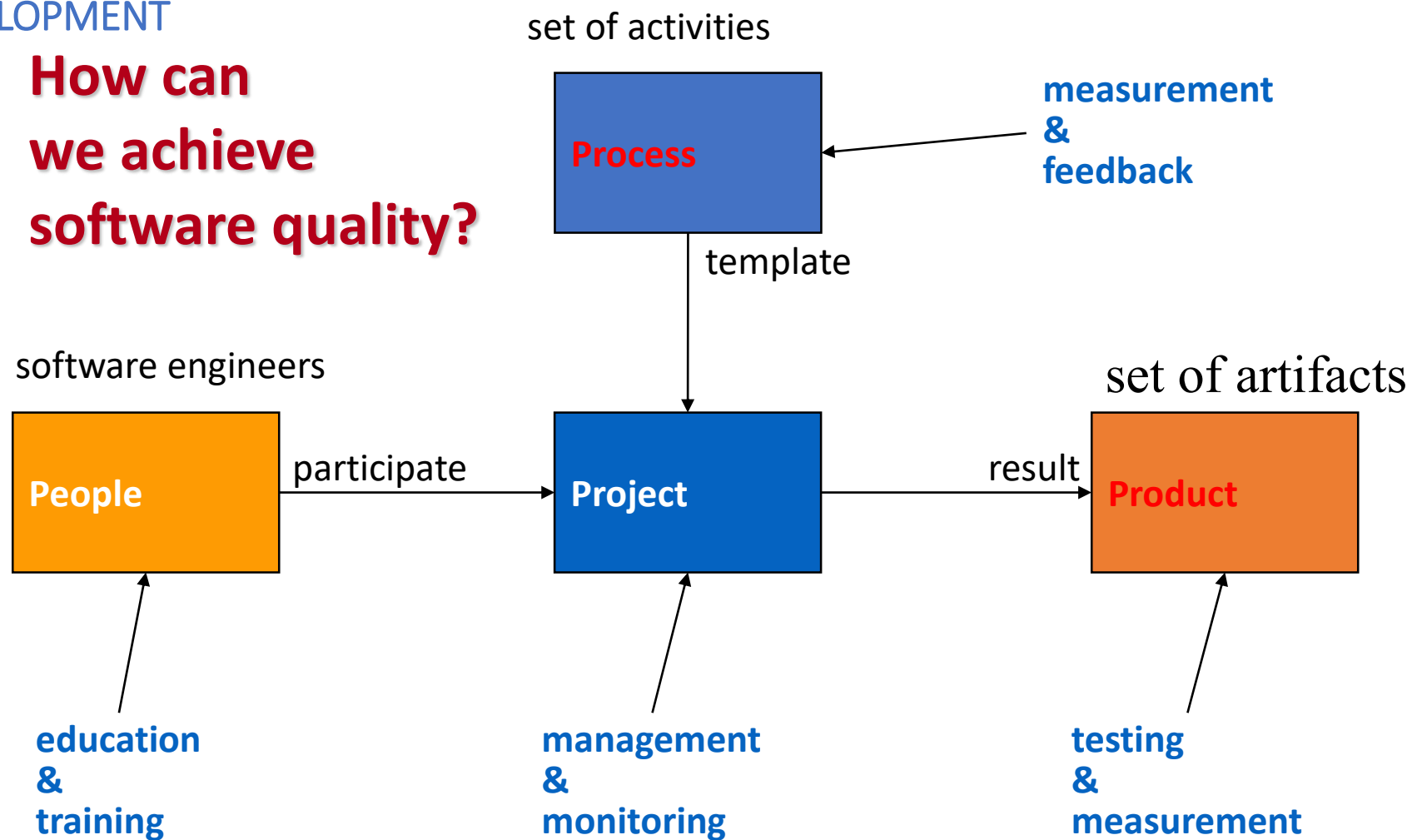


Chapter 1-2. SQA and SQA Introduction



THE FOUR P's IN SOFTWARE DEVELOPMENT

**How can
we achieve
software quality?**



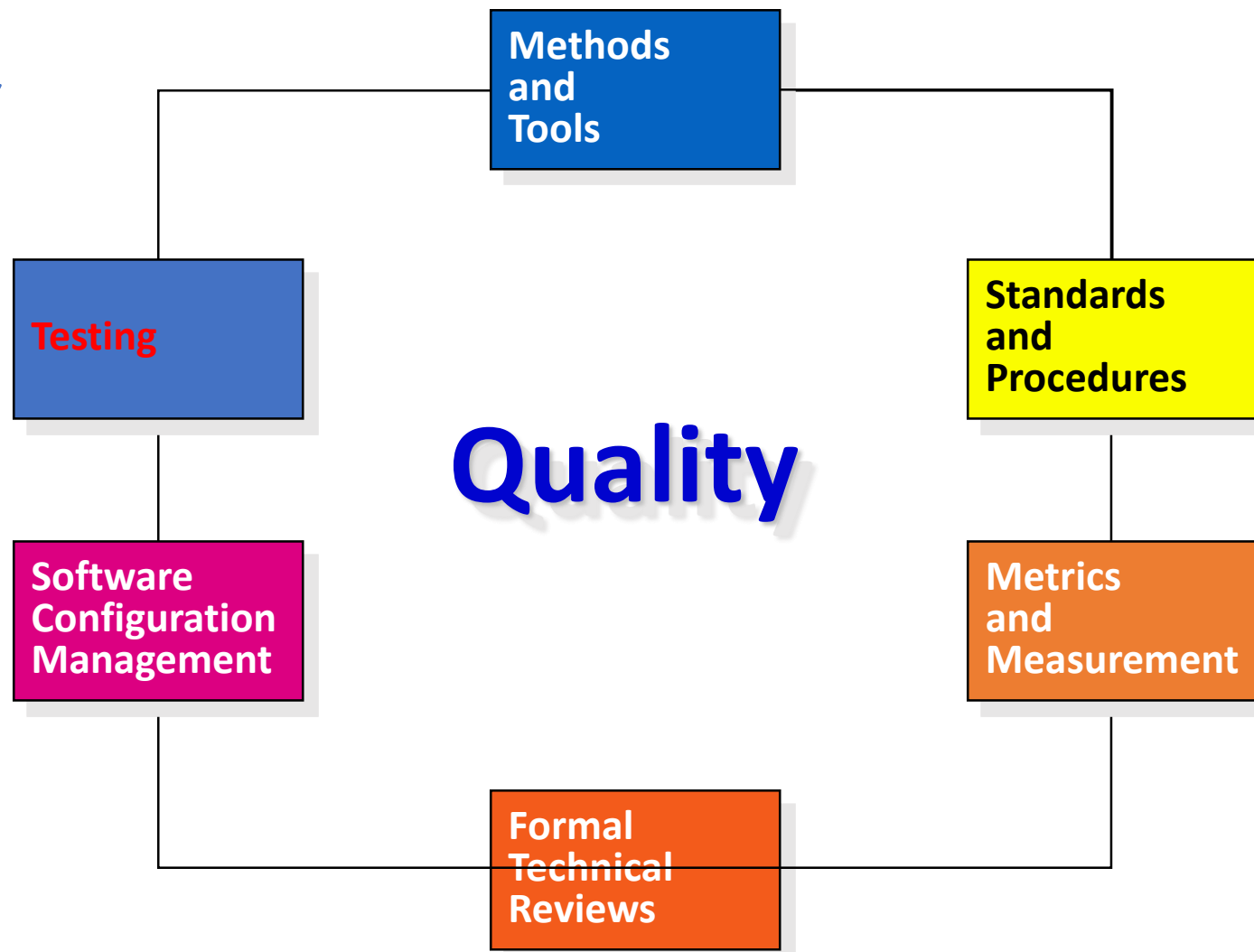
SQA

Quality assurance consists of those procedures, techniques, and tools applied by professionals to ensure that a product meets or exceeds pre-specified standards during it's development cycle.

E.H. Bersoff

- It is an **essential activity** for any business that produces products used by others.
- It needs to be **planned** and **systematic**.
(It does not just happen)
- It needs to be **built into** the development process.
(A natural outcome of software engineering)
- **Continuous improvement** is the overall goal.

SQA — AN UMBRELLA ACTIVITY



WHY SQA ACTIVITIES PAY OFF

cost to find
and fix a defect

log
scale

100

10

1

Reqmts

Design

Code

Test

System
test

Field
use

1.00

1.30

2.00

4.00

13.00

80-130

SQA pays off here because
defects are cheap to fix



What is software quality and how do we measure it?

customer's viewpoint → meets specifications

developer's viewpoint → easy to maintain, test, . . .

➔ Software quality is not just about **meeting specifications** and **removing defects**!

SOFTWARE QUALITY

Other attributes of software that affect its quality:

- | | | |
|---------------|---------------------|----------------|
| – safety | – understandability | – portability |
| – security | – testability | – usability |
| – reliability | – adaptability | – reusability |
| – resilience | – modularity | – efficiency |
| – robustness | – complexity | – learnability |

We need to **select critical quality attributes** early in the development process and **plan for how to achieve them.**

PRINCIPLES OF SOFTWARE QUALITY ASSURANCE

1. We have a set of **standards** and **quality attributes** that a software product must meet.

→ **There is a goal to achieve.**

2. We can **measure the quality** of a software product.

← **key point**

→ **There is a way to determine how well the product conforms to the standards and the quality attributes.**

3. We **track** the values of the quality attributes.

→ **It is possible to assess how well we are doing.**

4. We use information about software quality to **improve the quality** of future software products.

→ **There is feedback into the software development process.**

SOFTWARE STANDARDS

Why are software standards important?

1. encapsulate **best** (or **most appropriate**) practices
 - acquired after much trial and error → helps avoid previous mistakes
 2. provide a **framework** around which to implement SQA process
 - ensures that best practices are properly followed
 3. assist in ensuring **continuity of project work**
 - reduces learning effort when starting new work
- **product standards**: define the characteristics all product artifacts should exhibit so as to have quality
 - **process standards**: define how the software process should be conducted to ensure quality software

Each project needs to decide which standards should be:
ignored; used as is; modified; created.

metric: any type of measurement that relates to a software system, process or related artifact

SOFTWARE METRICS

control metrics - used to **plan, manage and control** the development process
(e.g., effort expended, elapsed time, disk usage, etc.)

predictor metrics - used to **predict** an associated **product quality**
(e.g., **cyclomatic complexity** can predict **ease of maintenance**)

external attribute: something we can only discover **after** the software has been put into use (e.g., ease of maintenance)

internal attribute: something we can **measure directly** from the software itself (e.g., cyclomatic complexity)

We want to **use internal attributes to predict** the value of **external attributes**.

SOFTWARE METRICS (cont'd)

external

maintainability

reliability

portability

usability

internal

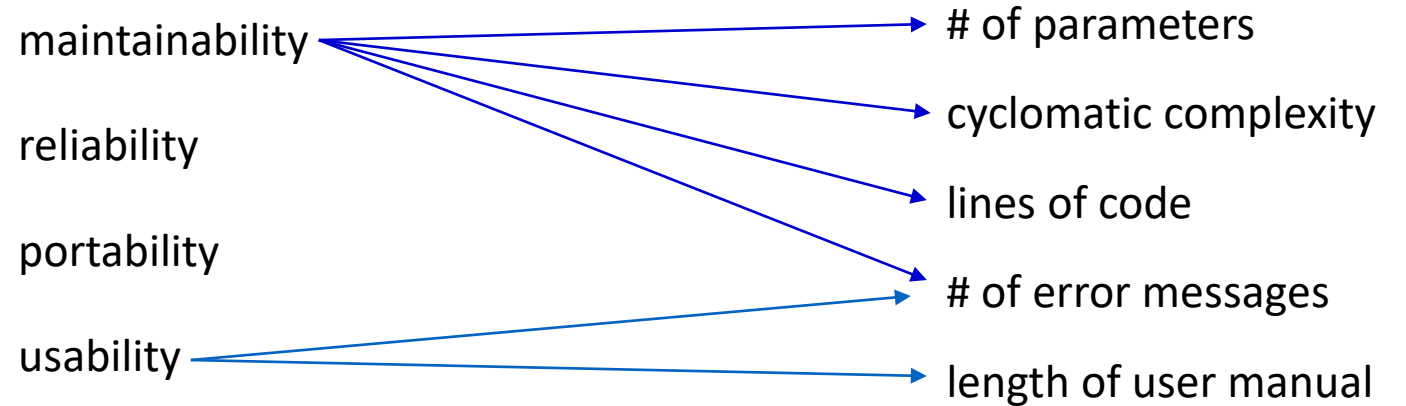
of parameters

cyclomatic complexity

lines of code

of error messages

length of user manual



- Problems:**
1. **hard to formulate** and **validate relationships** between internal and external attributes
 2. software metrics must be **collected, calibrated, interpreted**

For a design component, the key quality attribute is maintainability.

For design components maintainability is related to:

- **cohesion** - how closely related is the functionality of the component?
- **coupling** - how independent is the component?
- **understandability** - how easy is it to understand what the component does?
- **adaptability** - how easy is it to change the component?

Problem: most of these cannot be measured directly, but it is reasonable to infer that there is a relationship between these attributes and the “**complexity**” of a component

→ **measure complexity** **How?**



PRODUCT QUALITY: DESIGN QUALITY METRICS

a) Structural fan-in/fan-out

fan-in – number of calls to a component by other components

fan-out – number of components called by a component

→ high fan-in => high coupling

→ high fan-out => calling component has high complexity

b) Informational fan-in/fan-out

– consider also the **number of parameters** passed plus **access to shared data structures**

$\text{complexity} = \text{component-length} \times (\text{fan-in} \times \text{fan-out})^2$

→ It has been validated using the Unix system

→ It is a useful predictor of **effort** required for **implementation**

PRODUCT QUALITY: DESIGN QUALITY METRICS

c) IEEE Standard 982.1-1988

- looks at: **subsystem properties** (number of subsystems and degree of coupling)
database properties (number of attributes and classes)
 - compute a **design structure quality index—DSQI** → (0-1)
 - used to **compare with past designs**; if DSQI is too low, further design work and review may be required
- we can also consider **changes** made throughout the lifetime of the software and compute how **stable** the product is (i.e., how many changes have been made in subsystems in the current release)
 - define a **software maturity index—SMI** → (0-1)
 - as SMI approaches 1, the product begins to stabilize

PRODUCT QUALITY: FORMAL APPROACHES

a) Proving programs/specifications correct

- logically prove that requirements have been correctly transformed into programs
(e.g., prove assertions about programs)

b) Statistical Quality Assurance

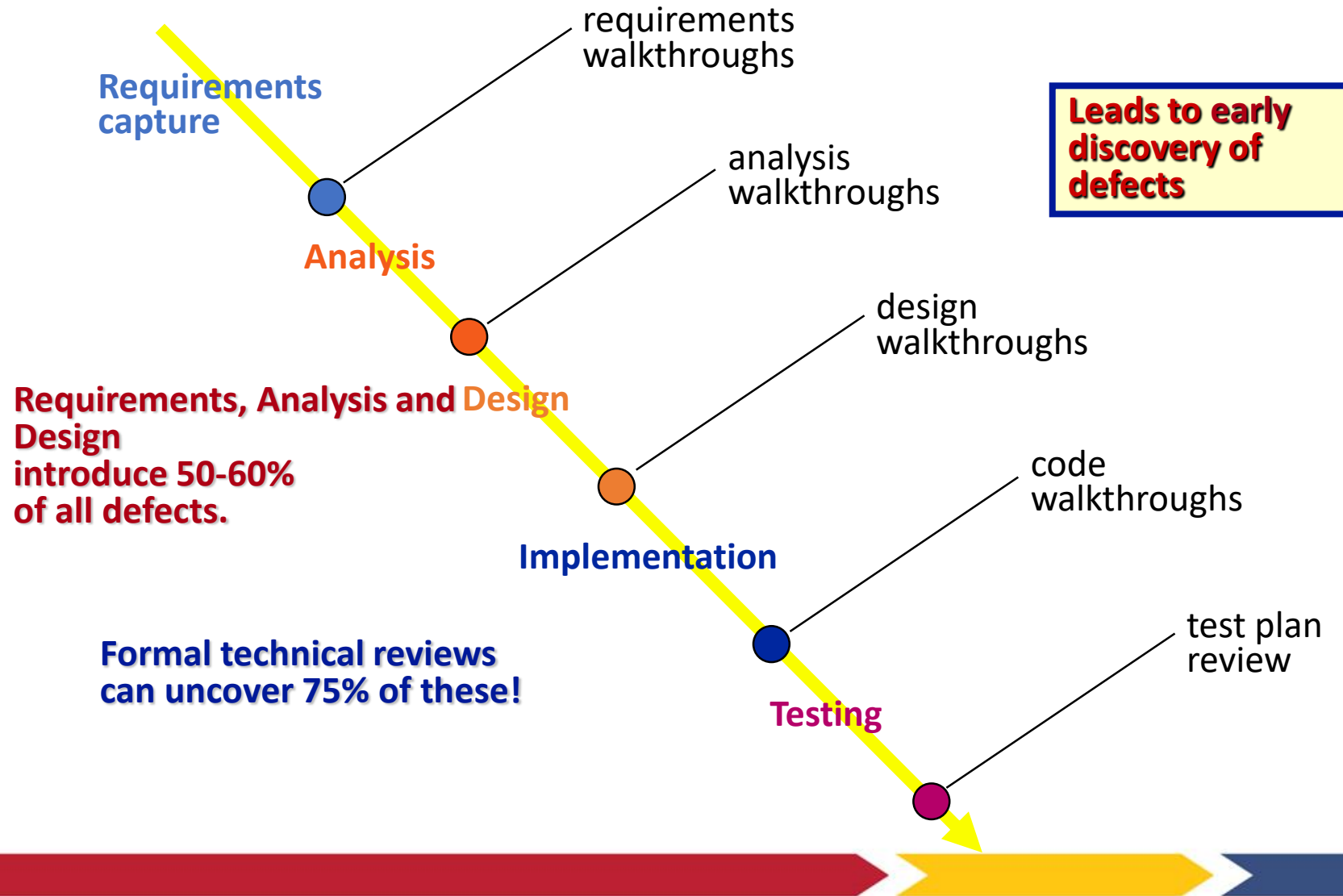
- categorize and determine cause of software defects
- **80-20 rule** → 80% of defects can be traced to 20% of causes
- isolate and correct 20% of the causes
 ➔ **effort directed to things that cause the majority of defects**

c) The Cleanroom Process

- combination of above two approaches

The principal method of validating the quality of a project.

PROJECT QUALITY: REVIEWS



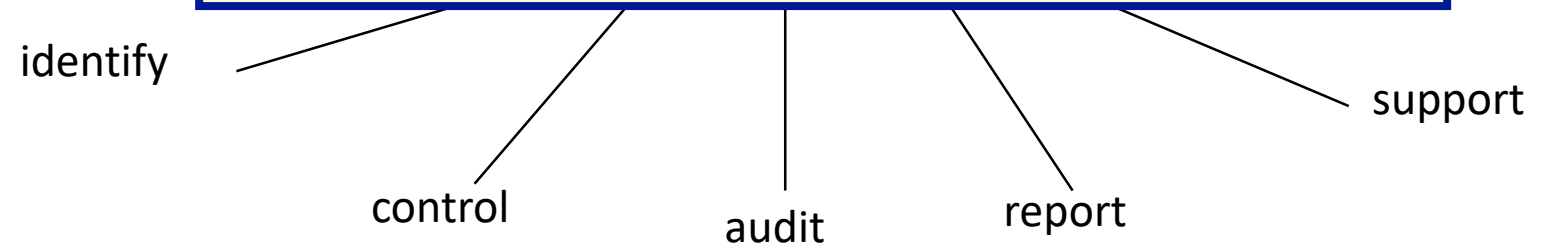
An activity executed throughout the system life cycle to control change of products and life cycle artifacts.

SOFTWARE CONFIGURATION MANAGEMENT (SCM)

To what do we want to control changes?

Software Configuration Management

- plans
- specifications
- procedures
- programs
- documents/manuals
- data



configuration item: an artifact to which we want to control changes

CONFIGURATION ITEM IDENTIFICATION AND DESCRIPTION

- each configuration item must be **identified** and **described**
 - a unique name
 - configuration item type
 - project identifier
 - change and/or version information
 - resources the configuration item provides or requires
 - pointer to actual configuration item
- the configuration items also must be **organized** (i.e., need to define the relationships that exist between configuration items)
 - object diagram <part-of> domain model
 - domain model <related-to> use-case model

**This is
metadata
about
configuration
items**

Define and construct a **software library (database)** that **stores, manages** and **tracks** configuration items.

CONTROLLING CONFIGURATION ITEMS: BASELINES

A baseline is a time/phase in the software development (usually a project milestone) after which any changes must be formalized (e.g., go through a formal change control procedure).

- In order to become a **baseline**, the configuration item must first pass a set of **formal review procedures** (e.g., formal code review, documentation review, etc.).
- It then becomes part of the project **software library**.
 - ➔ **After this a “check-out” procedure is applied to the item** (i.e., access to and change of the configuration item is controlled)
- Any modified configuration item must again go through a formal review process before it can replace the original (baselined) item.

CONTROLLING CONFIGURATION ITEMS: VERSION CONTROL

- a configuration item usually **evolves** throughout the software engineering process (i.e., it will have several **versions**)
 - An **evolution graph** can be used to describe a configuration item's change history

version

- configuration item_k is obtained by **modifying** configuration item_i
- usually a result of bug fixes, enhanced system functionality, etc.
- item_k **supercedes original** item_i; **created in a linear order**

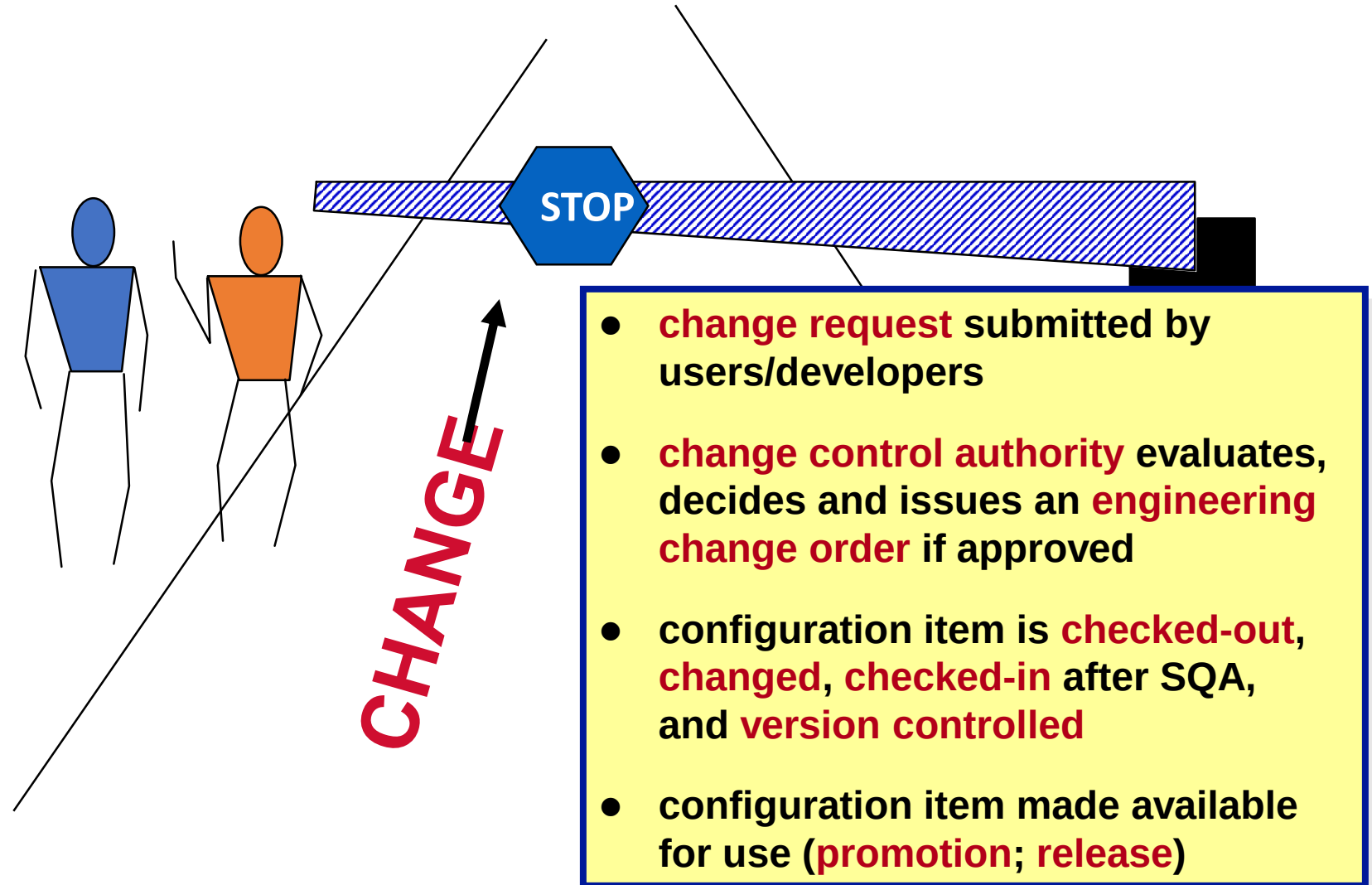
branch

- a **concurrent development path** requiring independent configuration management

variant

- **different configurations** that are intended to **coexist**
- e.g., different configurations depending on operating system type
 - Oracle for Windows
 - Oracle for Linux

SCM CHANGE CONTROL



SCM AUDIT AND STATUS REPORTING

Audit: ensures that changes have been properly implemented.

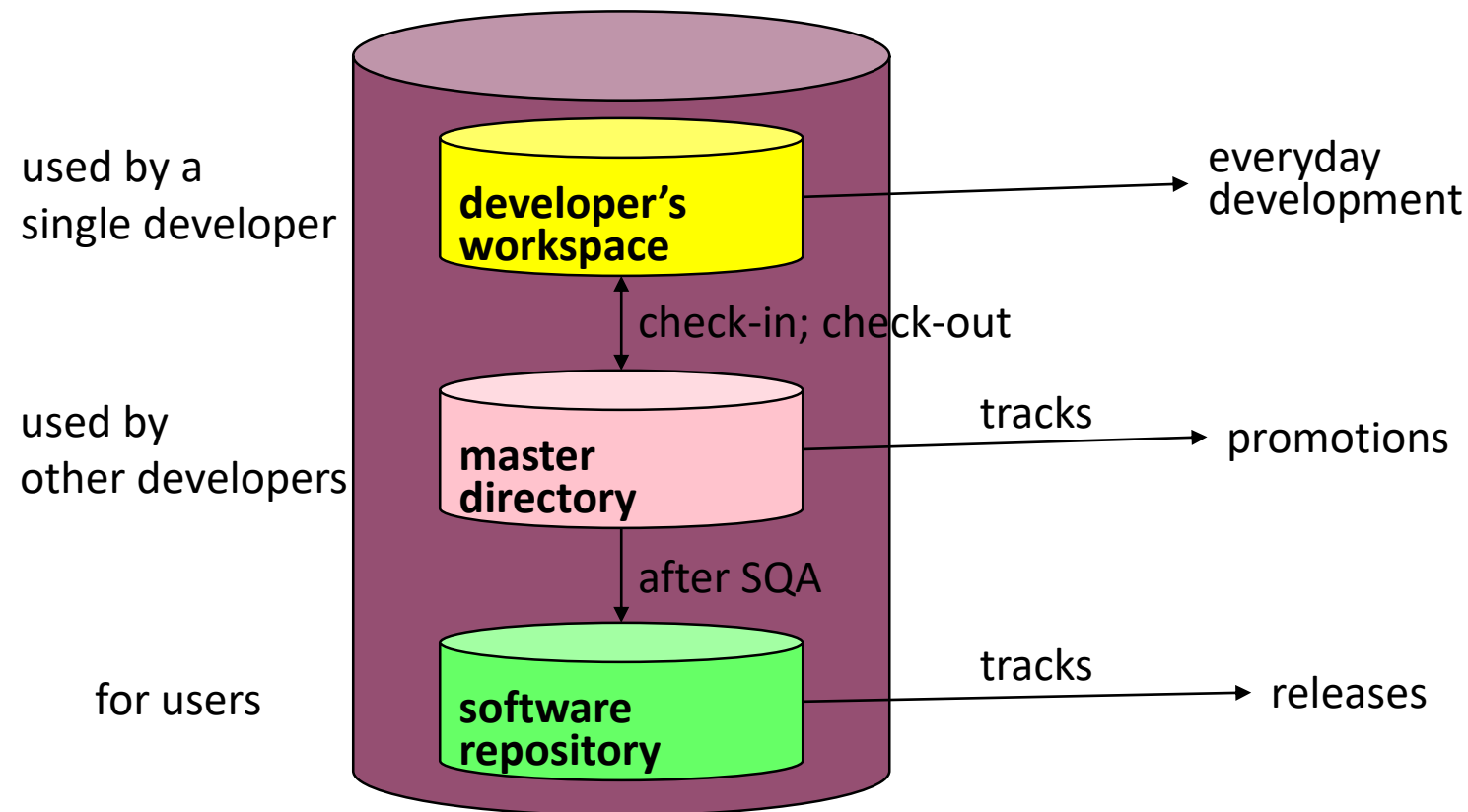
- Have the proper steps and procedures been followed? → checklist
- Usually done by Quality Assurance (QA) group if SCM is a formal activity.

Status reporting: keeps all parties informed and up-to-date on the status of a change.

- A communication mechanism among project members to help keep them coordinated.
- Can determine who made what changes, when and why.

SCM SUPPORT

- a **software library** provides facilities to store, label, identify versions and track the status of the configuration items



SCM BENEFITS

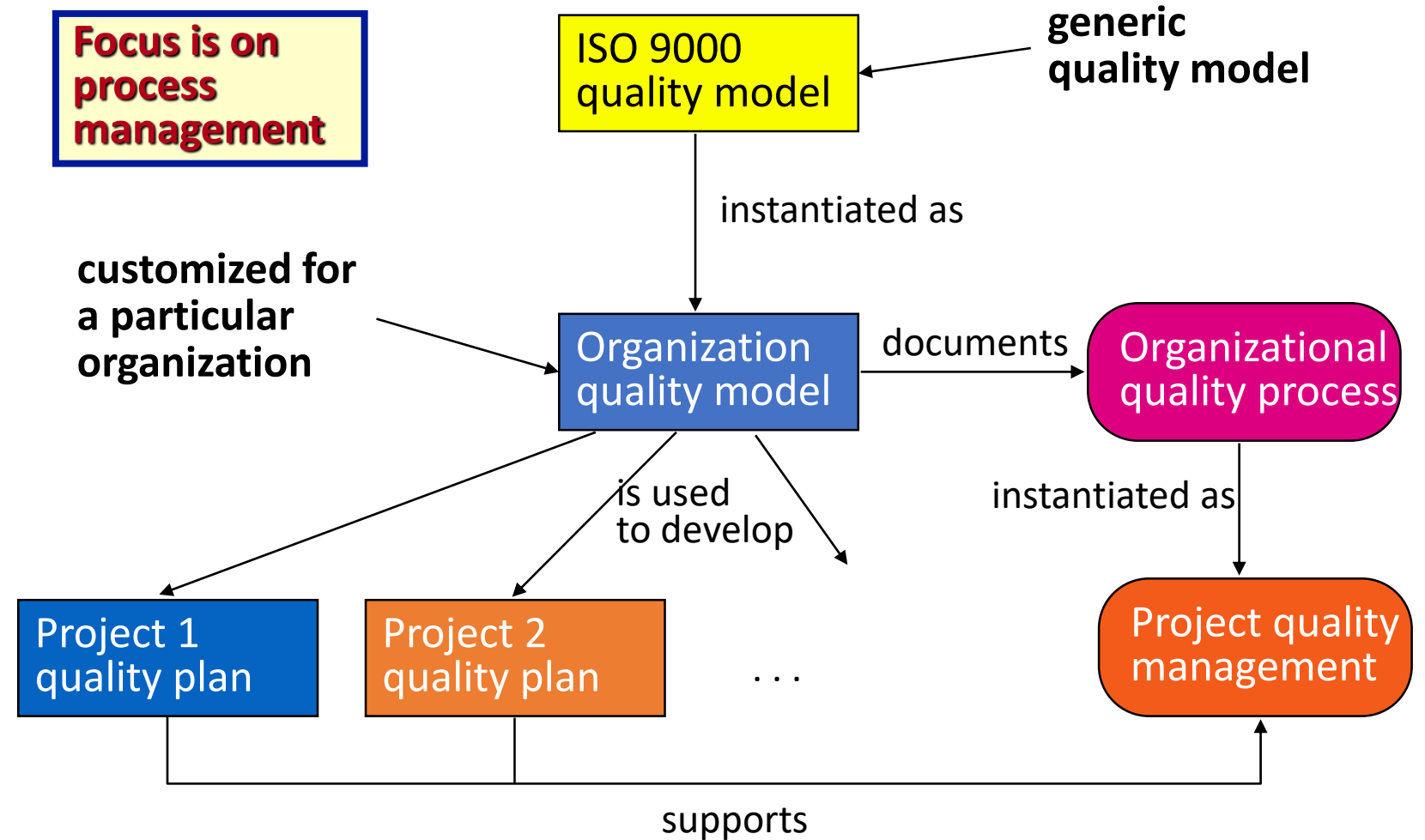
- reduces the effort required to manage and effect change
 - ➔ **improved productivity**
- leads to better software integrity and security
 - ➔ **increased quality**
- generates information about the process
 - ➔ **enhanced management control**
- maintains a software development database
 - ➔ **better record keeping and tracking**

Does process quality → product quality?

PROCESS QUALITY

- unlike manufactured products, software development has some unique factors that affect its quality:
 - software is **designed**, not manufactured
 - software development is **creative**, not mechanical
 - individual **skills** and **experience** have a significant influence
 - **external factors** (application novelty, commercial pressure)
- ➔ software development processes are **organization specific**
- ➔ **people, technology** may be more important than process
- ➔ **insufficient resources** will always adversely affect quality

PROCESS QUALITY: ISO 9001/ 9000-3



➔ certification is easy; can be a **marketing play**

PROCESS QUALITY: THE SEI CAPABILITY MATURITY MODEL (CMM)

Intended to help a software organization improve their software development processes.

Level 1 Organization: Initial process (ad hoc)

- no formal procedures, no cost estimates, no project plans, no management mechanism to ensure procedures are followed

Level 2 Organization: Repeatable process (intuitive)

- basic project controls; intuitive methods used

Level 3 Organization: Defined process (qualitative)

- development process defined and institutionalized

Level 4 Organization: Managed process (quantitative)

- measured process; process database established

Level 5 Organization: Optimizing process

- improvement feedback; rigorous defect-cause analysis and prevention

**Focus is on
process
improvement**

PEOPLE QUALITY: PEOPLE CAPABILITY MATURITY MODEL (PCMM)

Intended to improve knowledge and skill of people.

Level 1 – Initial

- no technical or management training provided; staff talent not a critical resource; no organizational loyalty

Focus is on
people
improvement

Level 2 – Repeatable

- focus on **developing basic work practices**; staff recruiting, growth and development important; training to fill skill “gaps”; performance evaluated

Level 3 – Defined

- focus on **tailoring work practices** to organization’s business; strategic plan to locate and develop required talent; skills-based compensation

Level 4 – Managed

- focus on **increasing competence in critical skills**; mentoring; team-building; quantitative competence goals; evaluation of effectiveness of work practices

Level 5 – Optimizing

- focus on **improving team and individual skills**; use of best practices

Quality software does not just happen!

SUMMARY- SOFTWARE QUALITY

- Quality assurance mechanisms should be built into the software development process
- Developing quality software requires:
 - Management support and involvement
 - Gathering and use of software metrics
 - Policies and procedures that everyone follows
 - Commitment to following the policies and procedures even when things get rough!

Testing is an important part of quality assurance, but its not all there is to obtaining a quality software product.



Chapter 3. SQA Management



What is Software Quality?

- Simplistically, quality is an attribute of software that implies the software meets its specification.
- This definition is too simple for ensuring quality in software systems
 - Software specifications are often incomplete or ambiguous
 - Some quality attributes are difficult to specify
 - Tension exists between some quality attributes, e.g. efficiency vs. reliability



What is Software Quality?

- Conformance to explicitly stated functional and performance requirements, explicitly documented development standards, and implicit characteristics that are expected of all professionally developed software
 - Software requirements are the foundation from which quality is measured.
 - Lack of conformance to requirements is lack of quality.
 - Specified standards define a set of development criteria that guide the manner in which software is engineered.
 - If the criteria are not met, lack of quality will almost surely result.
 - There is a set of implicit requirements that often goes unmentioned.
 - If software conforms to its explicit requirements but fails to meet its implicit requirements, software quality is suspect.



Software Quality Assurance - SQA

- To ensure quality in a software product, an organization must have a three-prong approach to quality management:
 - Organization-wide policies, procedures and standards must be established.
 - Project-specific policies, procedures and standards must be tailored from the organization-wide templates.
 - Quality must be controlled; that is, the organization must ensure that the appropriate procedures are followed for each project
- Standards exist to help an organization draft an appropriate software quality assurance plan.
 - ISO 9000-3
 - ANSI/IEEE standards
- External entities can be contracted to verify that an organization is standard-compliant.

SQA Activities

- Applying technical methods
 - To help the analyst achieve a high quality specification and a high quality design
- Conducting formal technical reviews
 - A stylized meeting conducted by technical staff with the sole purpose of uncovering quality problems
- Testing Software
 - A series of test case design methods that help ensure effective error detection
- Enforcing standards
- Controlling change
 - Applied during software development and maintenance
- Measurement
 - Track software quality and asses the ability of methodological and procedural changes to improve software quality
- Record keeping and reporting
 - Provide procedures for the collection and dissemination of SQA information

SQA Advantages

- Software will have fewer latent defects, resulting in reduced effort and time spent during testing and maintenance.
- Higher reliability will result in greater customer satisfaction.
- Maintenance costs can be reduced.
- Overall life cycle cost of software is reduced.



SQA Disadvantages

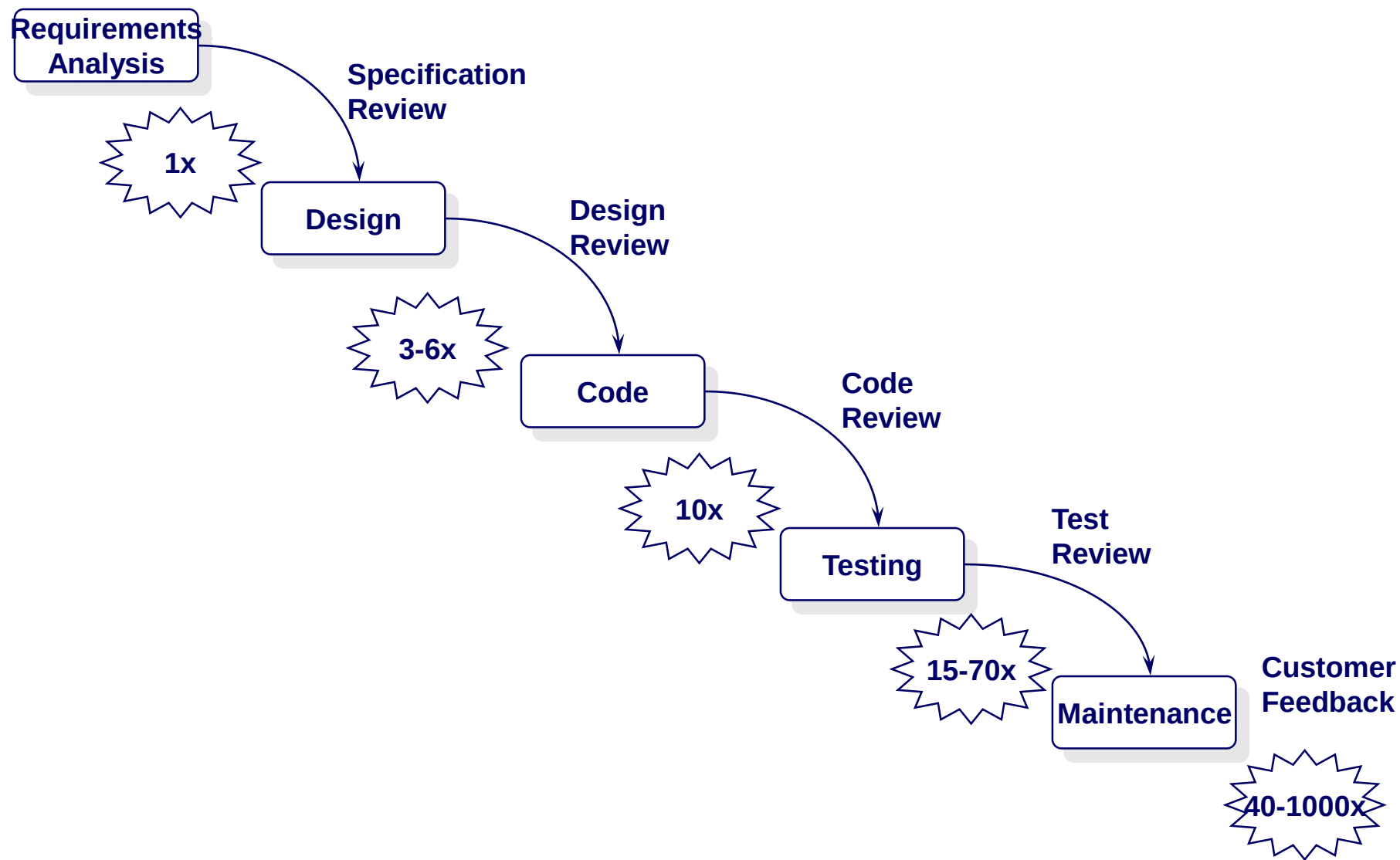
- It is difficult to institute in small organizations, where available resources to perform necessary activities are not available.
- It represents cultural change - and change is never easy.
- It requires the expenditure of dollars that would not otherwise be explicitly budgeted to software engineering or QA.



Quality Reviews

- The fundamental method of validating the quality of a product or a process.
- Applied during and/or at the end of each life cycle phase
 - Point out needed improvements in the product of a single person or team
 - Confirm those parts of a product in which improvement is either not desired or not needed
 - Achieve technical work of more uniform, or at least more predictable, quality than what can be achieved without reviews, in order to make technical work more manageable
- Quality reviews can have different intents:
 - review for defect removal
 - review for progress assessment
 - review for consistency and conformance

Quality Reviews



Cost Impact of Software Defects

Errors
from
Previous
Steps

Errors Passed Through	Percent Efficiency for error detection
Amplified Errors 1:X	
Newly Generated Errors	

Errors
Passed to
Next Step

Defect Amplification and Removal

Preliminary Design

0	0%
0	
10	

10

Detailed Design

6	0%
4x1.5	
25	

37

Code/Unit Testing

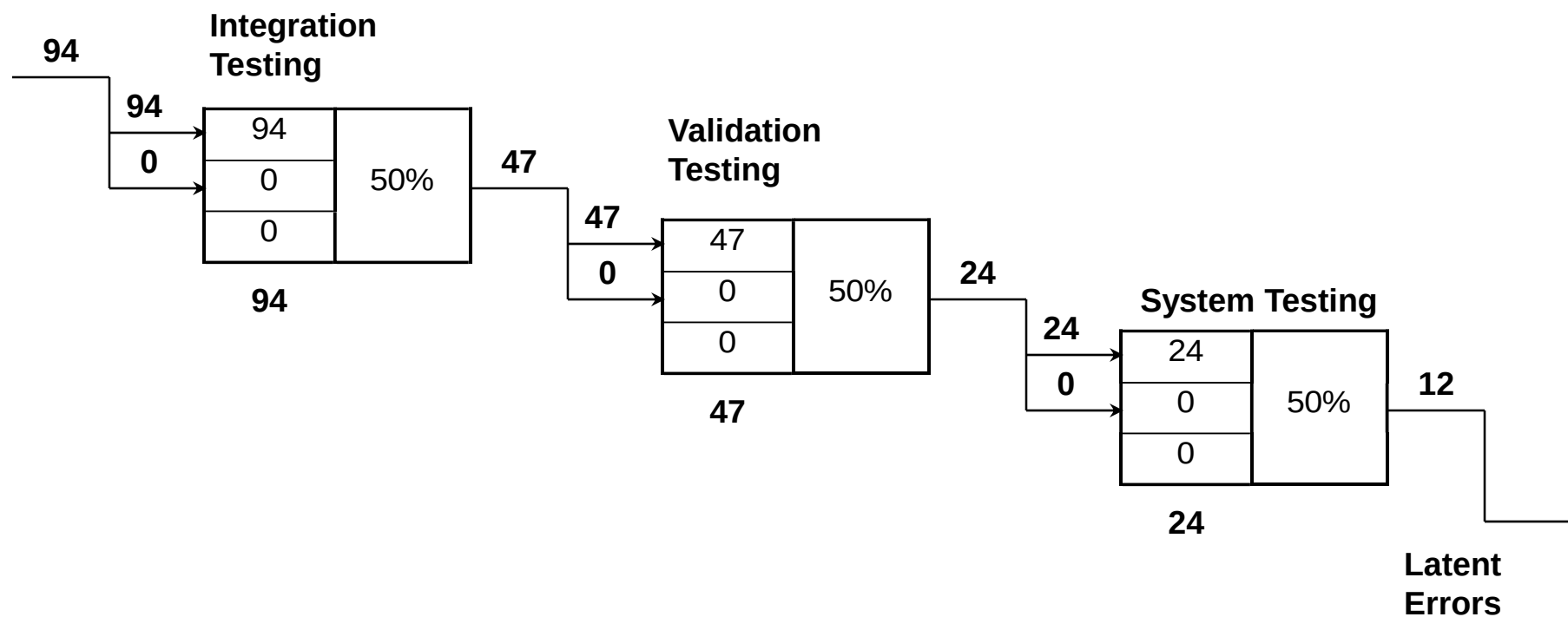
10	20%
27x3	
25	

116

To
integration
testing...



Defect Amplification and Removal (cont'd)



Review Checklist for System Engineering

- Are major functions defined in a bounded and unambiguous fashion?
- Are interfaces between system elements defined?
- Are performance bounds established for the system as a whole and for each element?
- Are design constraints established for each element?
- Has the best alternative been selected?
- Is the solution technologically feasible?
- Has a mechanism for system validation and verification been established?
- Is there consistency among all system elements?

Review Checklist for Software Project Planning

- Is the software scope unambiguously defined and bounded?
- Is terminology clear?
- Are resources adequate for the scope?
- Are resources readily available?
- Are tasks properly defined and sequenced?
- Is the basis for cost estimation reasonable? Has it been developed using two different sources?
- Have historical productivity and quality data been used?
- Have differences in estimates been reconciled?
- Are pre-established budgets and deadlines realistic?
- Is the schedule consistent?

Review Checklist for Software Requirements Analysis

- Is the information domain analysis complete, consistent, and accurate?
- Is problem partitioning complete?
- Are external and internal interfaces properly defined?
- Are all requirements traceable to the system level?
- Is prototyping conducted for the customer?
- Is performance achievable with constraints imposed by other system elements?
- Are requirements consistent with schedule, resources, and budget?
- Are validation criteria complete?

Review Checklist for Software Design (Preliminary Design Review)

- Are software requirements reflected in the software architecture?
- Is effective modularity achieved? Are modules functionally independent?
- Is program architecture factored?
- Are interfaces defined for modules and external system elements?
- Is data structure consistent with software requirements?
- Has maintainability been considered?



Review Checklist for Software Design (Design Walkthrough)

- Does the algorithm accomplish the desired function?
- Is the algorithm logically correct?
- Is the interface consistent with architectural design?
- Is logical complexity reasonable?
- Have error handling and “antibugging” been specified?
- Is local data structure properly defined?
- Are structured programming constructs used throughout?
- Is design detail amenable to the implementation language?
- Which are used: operating system or language dependent features?
- Is compound or inverse logic used?
- Has maintainability been considered?



Review Checklist for Coding

- Is the design properly translated into code? (The results of the procedural design should be available at this review)
- Are there misspellings or typos?
- Has proper use of language conventions been made?
- Is there compliance with coding standards for language style, comments, module prologue?
- Are incorrect or ambiguous comments present?
- Are typing and data declaration proper?
- Are physical constraints correct?
- Have all items on the design walkthrough checklist been reapplied (as required)?



Review Checklist for Software Testing (Test Plan)

- Have major test phases been properly identified and sequenced?
- Has traceability to validation criteria/requirements been established as part of software requirements analysis?
- Are major functions demonstrated early?
- Is the test plan consistent with the overall project plan?
- Has a test schedule been explicitly defined?
- Are test resources and tools identified and available?
- Has a test recordkeeping mechanism been established?
- Have test drivers and stubs been identified, and has work to develop them been scheduled?
- Has stress testing for software been specified?

Review Checklist for Software Testing (Test Procedure)

- Have both white and black box tests been specified?
- Have all independent logic paths been tested?
- Have test cases been identified and listed with expected results?
- Is error handling to be tested?
- Are boundary values to be tested?
- Are timing and performance to be tested?
- Has acceptable variation from expected results been specified?



Review Checklist for Maintenance

- Have side effects associated with change been considered?
- Has the request for change been documented, evaluated, and approved?
- Has the change, once made, been documented and reported to interested parties?
- Have appropriate FTRs been conducted?
- Has a final acceptance review been conducted to assure that all software has been properly updated, tested, and replaced?



Formal Technical Review (FTR)

- Software quality assurance activity that is performed by software engineering practitioners
 - Uncover errors in function, logic, or implementation for any representation of the software
 - Verify that the software under review meets its requirements
 - Assure that the software has been represented according to predefined standards
 - Achieve software that is developed in a uniform manner
 - Make projects more manageable
- FTR is actually a class of reviews
 - Walkthroughs
 - Inspections
 - Round-robin reviews
 - Other small group technical assessments of the software

The Review Meeting

- Constraints
 - Between 3 and 5 people (typically) are involved
 - Advance preparation should occur, but should involve no more than 2 hours of work for each person
 - Duration should be less than two hours
- Components
 - Product - A component of software to be reviewed
 - Producer - The individual who developed the product
 - Review leader - Appointed by the project leader; evaluates the product for readiness, generates copies of product materials, and distributes them to 2 or 3 reviewers
 - Reviewers - Spend between 1 and 2 hours reviewing the product, making notes, and otherwise becoming familiar with the work
 - Recorder - The individual who records (in writing) all important issues raised during the review

Review Reporting and Recordkeeping

- Review Summary Report
 - What was reviewed?
 - Who reviewed it?
 - What were the findings and conclusions?
- Review Issues List
 - Identify the problem areas within the product
 - Serve as an action item checklist that guides the producer as corrections are made

Guidelines for FTR

- Review the product, not the producer
- Set an agenda and maintain it
- Limit debate and rebuttal
- Enunciate the problem areas, but don't attempt to solve every problem that is noted
- Take written notes
- Limit the number of participants and insist upon advance preparation
- Develop a checklist for each product that is likely to be reviewed
- Allocate resources and time schedules for FTRs
- Conduct meaningful training for all reviewers
- Review your earlier reviews (if any)



Reviewer's Preparation

- Be sure that you understand the context of the material
- Skim all product material to understand the location and the format of information
- Read the product material and annotate a hardcopy
- Pose your written comments as questions
- Avoid issues of style
- Inform the review leader if you cannot prepare

Results of Review Meeting

- All attendees of the FTR must make a decision
 - Accept the product without further modification
 - Reject the product due to severe errors (and perform another review after corrections have been made)
 - Accept the product provisionally (minor corrections are needed, but no further reviews are required)
- A sign-off is completed, indicating participation and concurrence with the review team's findings



Software Reliability

- Probability of failure-free operation for a specified time in a specified environment.
- This could mean very different things for different systems and different users.
- Informally, reliability is a measure of the users' perception of how well the software provides the services they need.
 - Not an objective measure
 - Must be based on an operational profile
 - Must consider that there are widely varying consequences for different errors

Software Reliability Improvements

- Software reliability improves when faults which are present in the most frequently used portions of the software are removed.
- A removal of X% of faults doesn't necessarily mean an X% improvement in reliability.
- In a study by Mills et al. in 1987 removing 60% of faults resulted in a 3% improvement in reliability.
- Removing faults with the most serious consequences is the primary objective.



SOFTWARE



SOFTWARE QUALITY ASSURANCE



Chapter 4. Introduction to Software Testing



Contents

- 1 What is software quality?
- 2 The cause of software errors
- 3 Principles of Testing
- 4 What is software testing? Testing objective and why Testing
- 5 Fundamental of testing process



What is software quality?

- Software quality – IEEE definition:

The degree to which a system, component, or process meets specified requirements.

CUSTOMER SATISFACTION



Error? Fault? Failure?

- **Error**

- Also known as mistake.
- An error can be a Syntax (grammatical) error or Logic
- A software error **made by human action** which produces an incorrect result.

- **Fault**

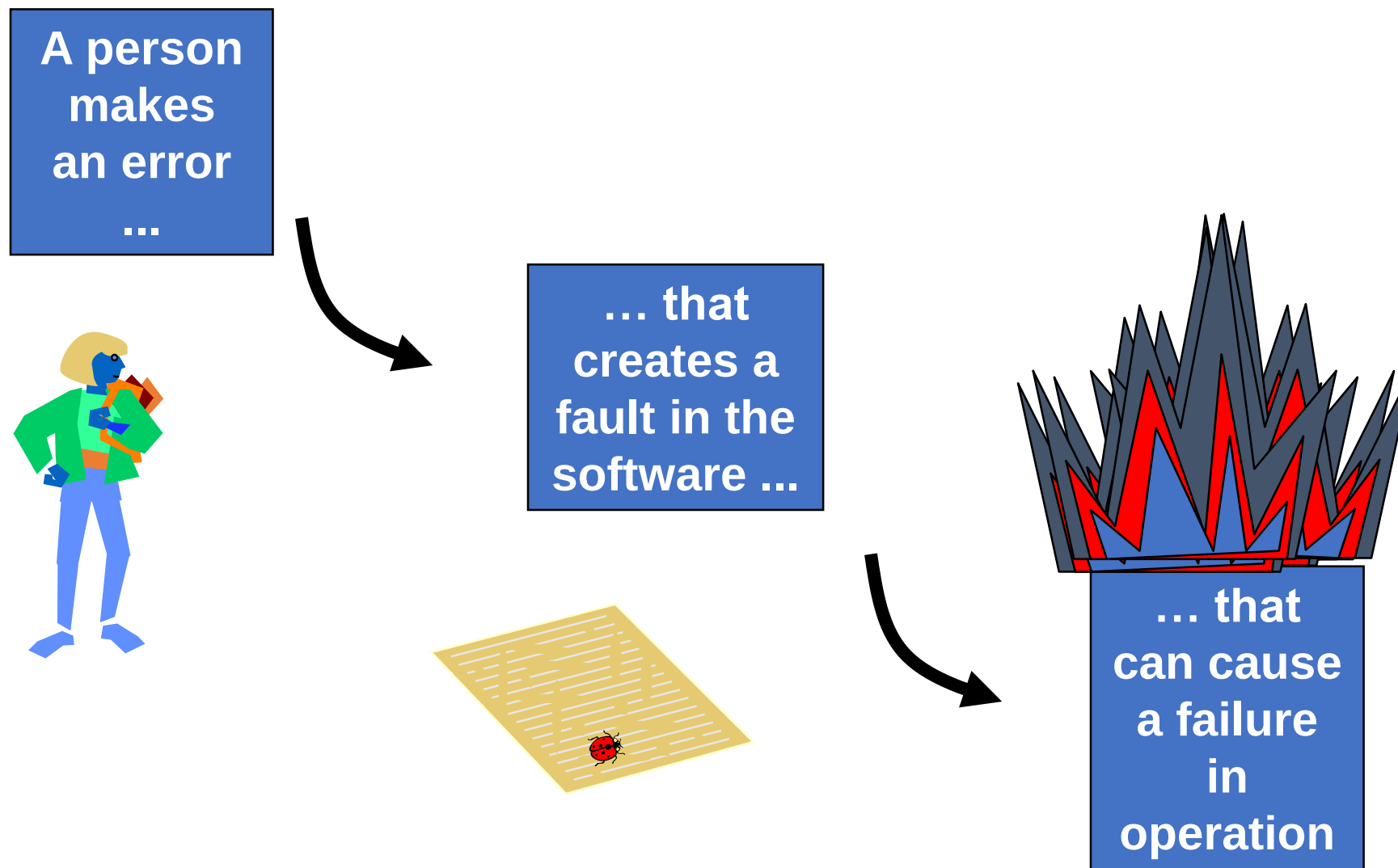
- Also known as a defect or bug.
- A fault is a manifestation of an error in software
- All software errors may **not** cause software faults

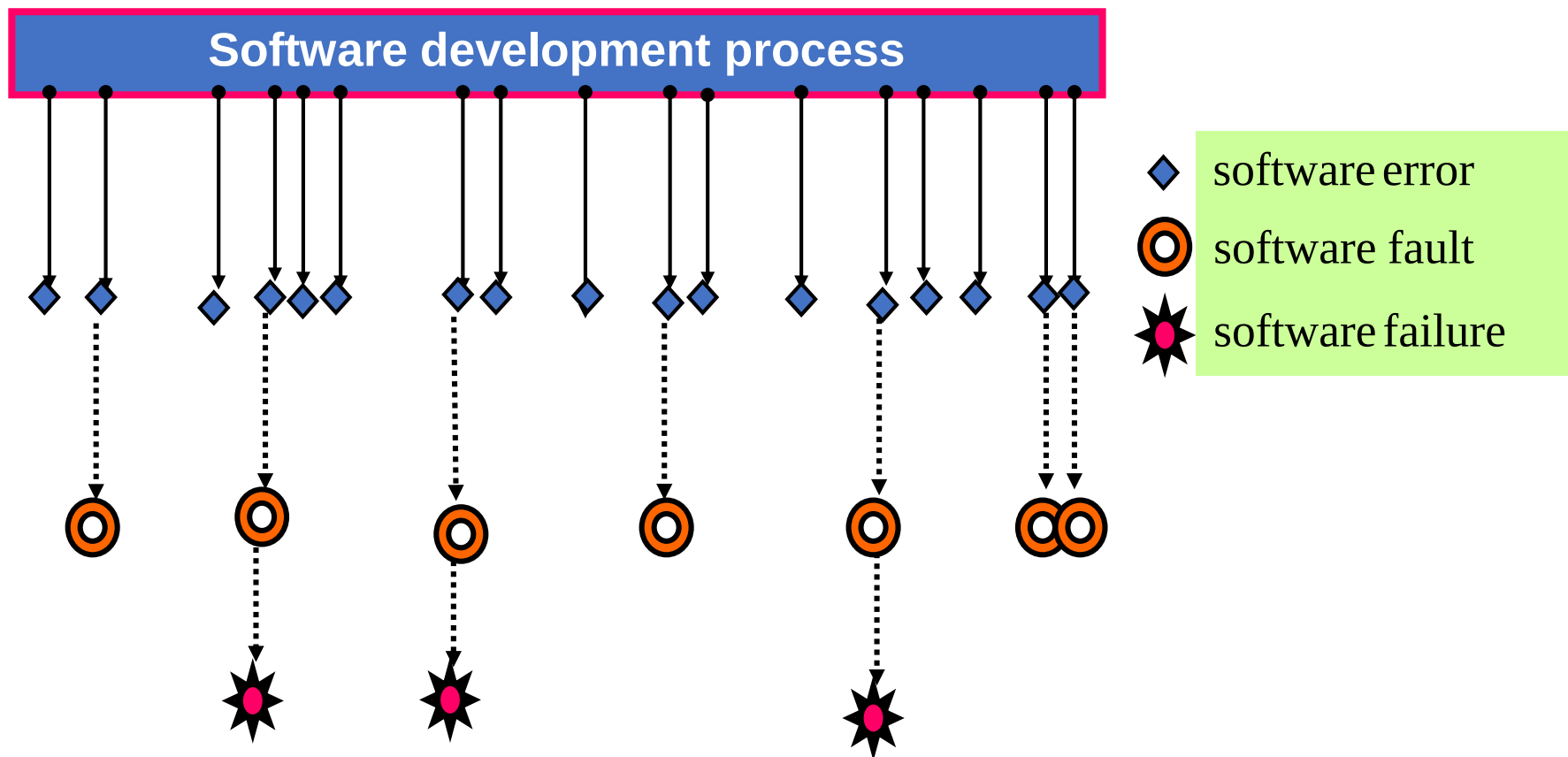
- **Failure**

- A fault becomes a failure if it is **activated/executed**.
 - deviation of the software from its expected delivery or service
 - Not all faults result in **failures**; some stay dormant in the code and we may never notice them.

Failure is an event;

Fault is a state of the software, caused by an error





Causes of software errors

- **Errors may occur for many reasons, such as:**
 - Time pressure
 - Human is error prone
 - Inexperienced or insufficiently skilled project participants
 - Miscommunication between project participants, including miscommunication about requirements and design
 - Complexity of the code, design, architecture, the underlying problem to be solved, and the technologies used
 - New, unfamiliar technologies

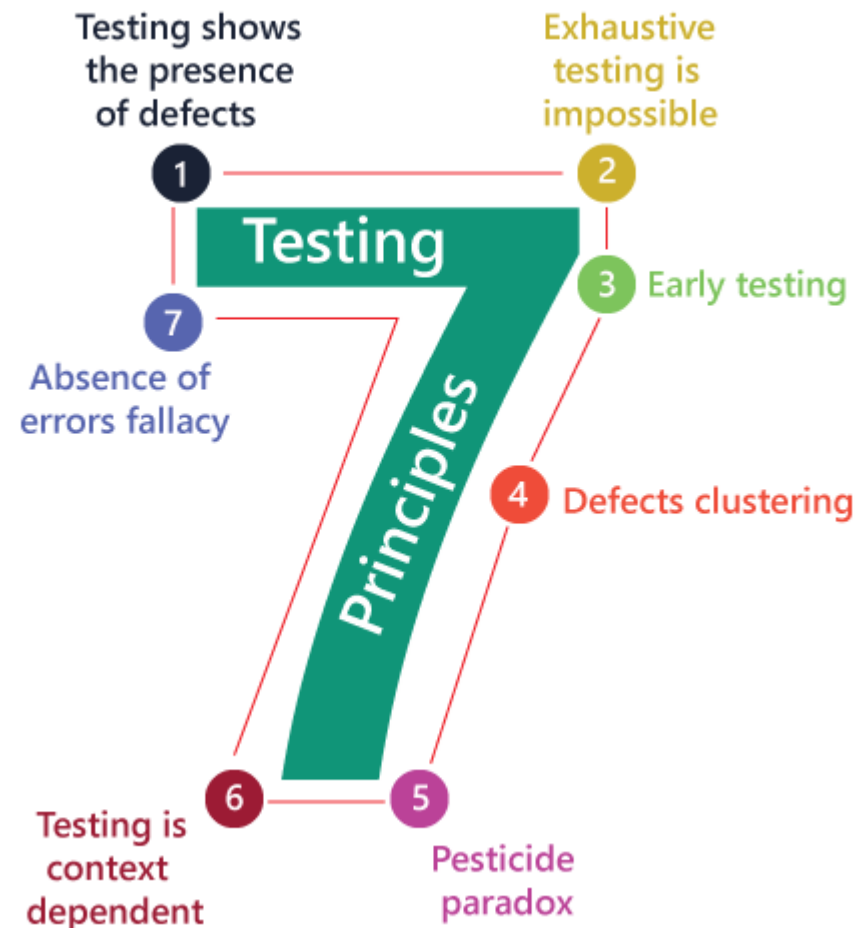


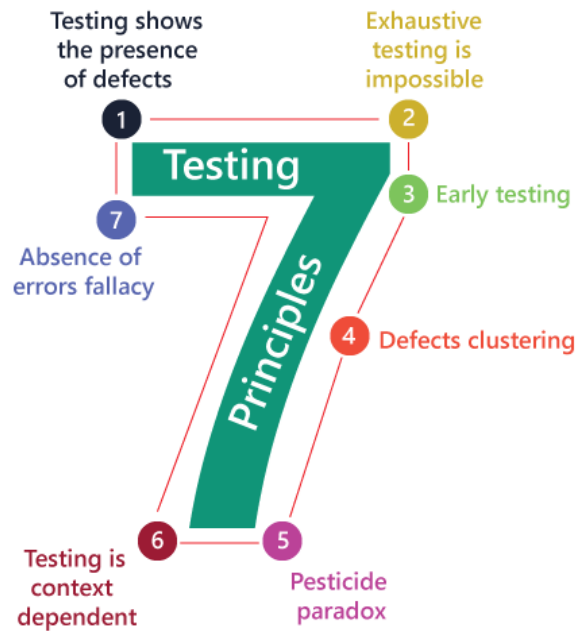
Definitions of Software Testing?

SOFTWARE TESTING

- Software Testing the **process** consisting of all lifecycle activities, both **static** and **dynamic**, concerned with planning, preparation and evaluation of software products and related work products to **determine** that they satisfy specified requirements, to **demonstrate** that they are fit for purpose and to **detect** defects.

Principles of Testing



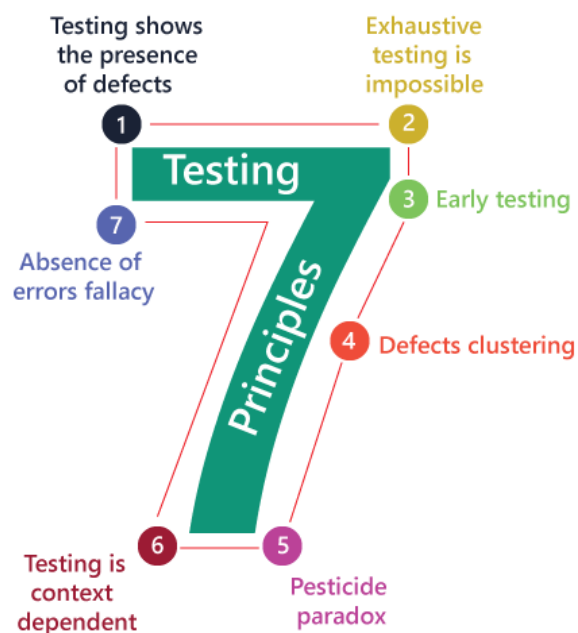


- Principle 1:



- Testing can show that defects are present , but cannot prove that there are no defects.

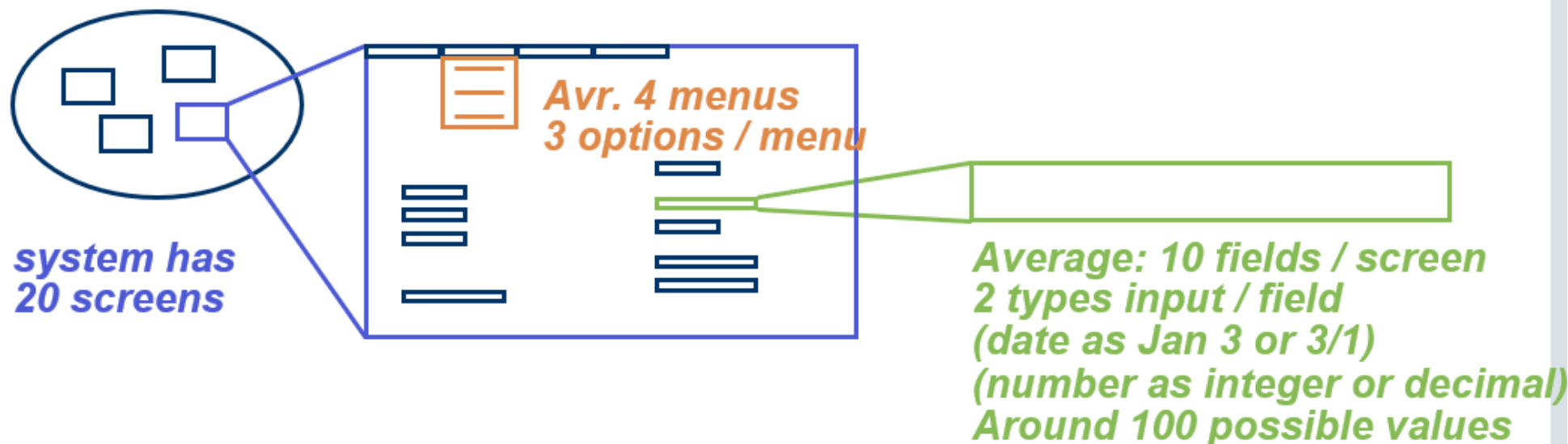
- Principle 2:



Exhaustive testing is impossible



- Exhaustive testing means to test everything, all preconditions and combinations of inputs.
- Testers should apply risk analysis and set priorities to focus testing efforts.



Total for 'exhaustive' testing:

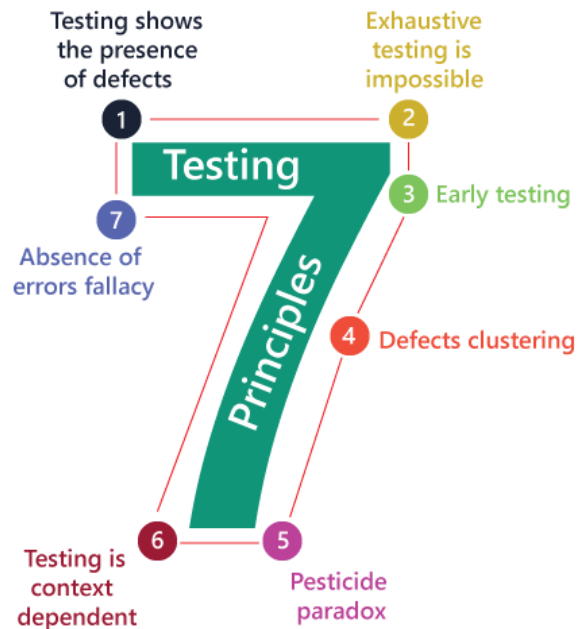
$$20 \times 4 \times 3 \times 10 \times 2 \times 100 = 480,000 \text{ tests}$$

If 1 second per test, 8000 mins, 133 hrs, 17.7 days

(not counting finger trouble, faults or retest)

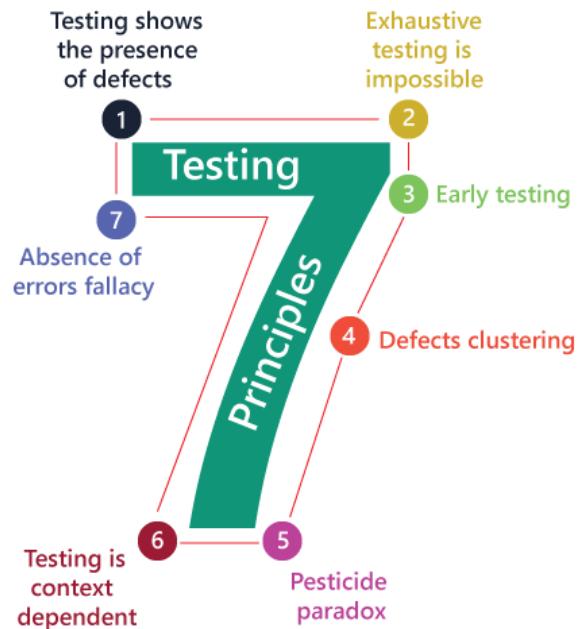
10 secs = 34 wks, 1 min = 4 yrs, 10 min = 40 yrs

- Principle 3:



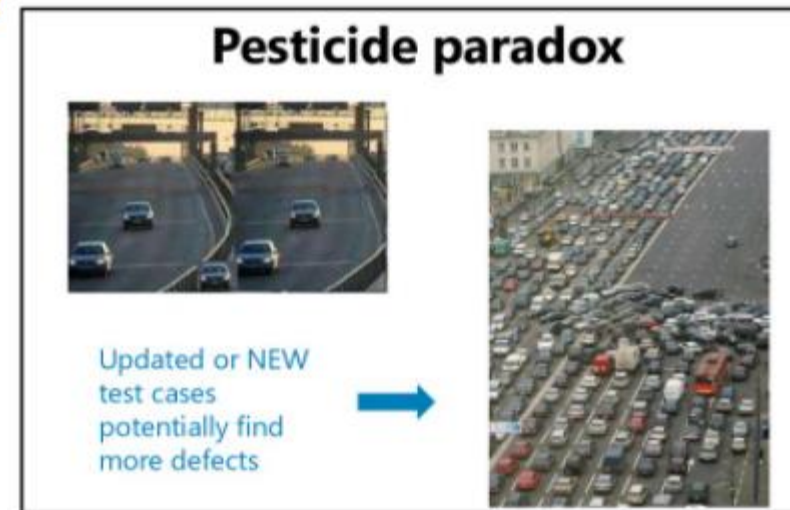
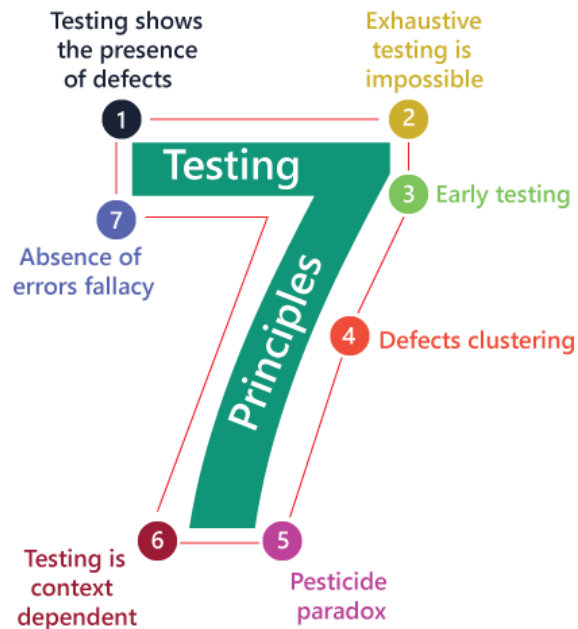
- Testing activities should start as early as possible in the software or system development life cycle and should be focused on defined objectives.

- Principle 4:



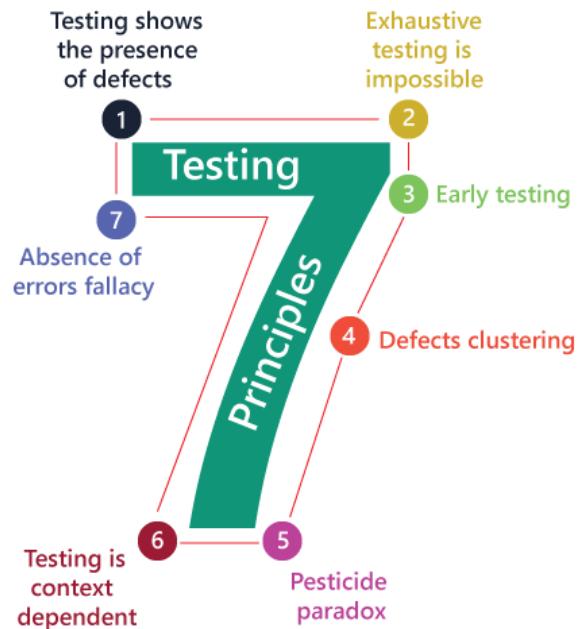
- A small number of modules contain most of the defects discovered during pre-release testing or show the most operational failures.

- Principle 5:



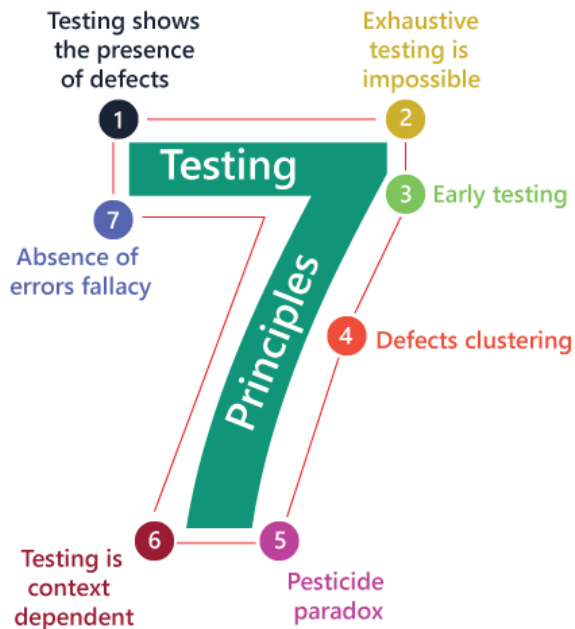
- If the same tests are repeated over and over again, eventually the same set of test cases will no longer find any new bugs.
- To overcome this 'pesticide paradox', the test cases need to be regularly reviewed and revised, and new and different tests need to be written to exercise different parts of the software or system to potentially find more defects.

- Principle 6:



- Testing is done differently in different dependent contexts.
- For example, safety-critical software is tested differently from an e-commerce site.

- Principle 7:



- Finding and fixing defects does not help if the system built is unusable and does not fulfill the users' needs and expectations.

Why Testing is necessary?

- All Software has defects (bugs)
- Software products are getting larger and more complicated
- Software is written by people – people make mistakes



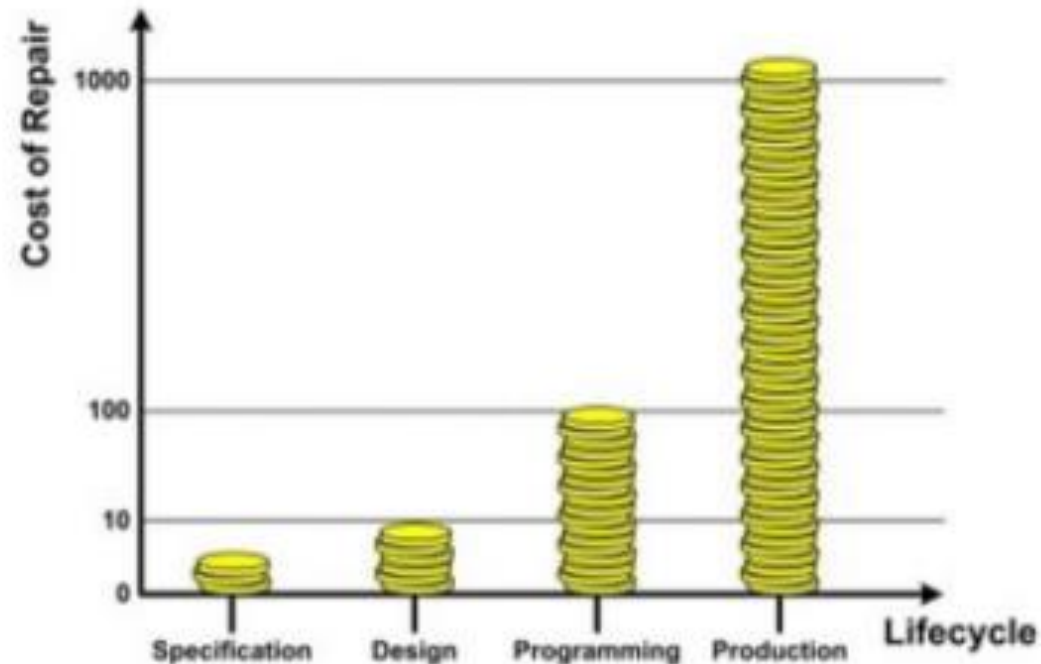
Why Testing is necessary?

- Some of the problems might be trivial, but others can be costly and damaging- with loss of money, time, or business reputation – and even may result in injury or death
- Not all software systems carry the same level of risk and not all problems have the same impact when they occur



Why Testing is necessary?

- Cost of defects: the cost to finding and fixing defects rises considerably across the life cycle



Why Testing is necessary?

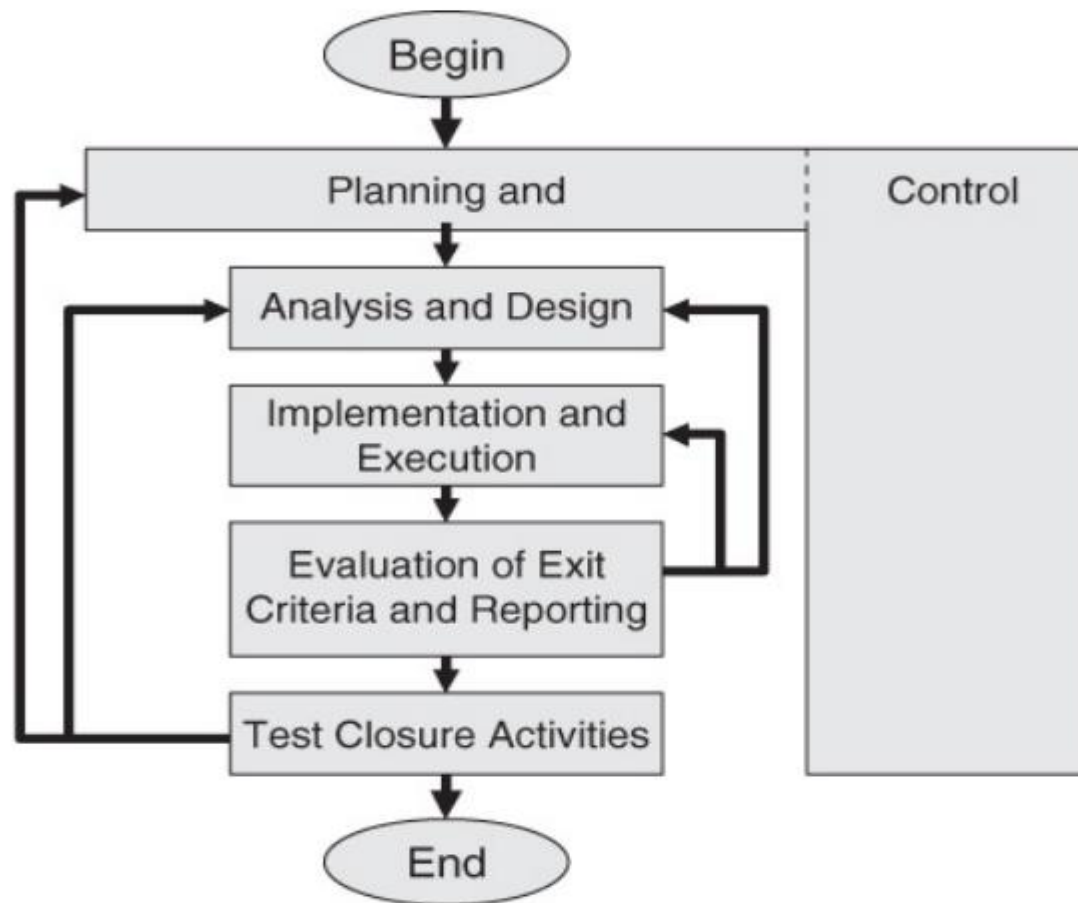
- Software testing looks to find the most important defects as early as possible – increasing confidence that the software meets specification
- Help to measure the quality of software



Software Testing Objectives

- To identify and reveal as many errors as possible
- To gain confidence about the *level of quality*
- To prevent defects
- To provide information for decision-making (stakeholders)
- To reduce the level of risk of inadequate of software quality
- To make sure that the system works as expected, meets the user requirements
- To comply with the contractual, legal or regulatory requirements or standards

- The five stages of the fundamental test process:



Test Planning

"A goal without a plan is just a wish"

- Major tasks are:
 - Identify the objectives of testing
 - Determine scope
 - Determine the Test Approach
 - Determine the required test resources
 - Implement the test policy and/or the test strategy
 - Schedule test analysis and design tasks
 - Schedule test implementation, execution and evaluation
 - Determine the Exit Criteria

Test Planning

- Test Control:
 - The ongoing activity of comparing actual progress against the plan
 - Reporting status, including deviations from the plan
 - Taking actions necessary to meet the mission and objectives of the project
 - Test Planning takes into account the feedback from monitoring and control activities
- Major tasks are:
 - Measure and analyze results
 - Monitor and document progress, test coverage and exit criteria
 - Initiate corrective actions
 - Make decisions

Analysis and design

"Analysis gain better understanding"

- Review the Test Basis: - in doing so evaluate testability of Test Basis and Test Objects(s).
- Identify and prioritize Test Conditions and associated Test Data.
- Test Conditions and associated Test Data are documented in a Test Design Specification.
- Design and prioritize the Test Cases
- Identify Test Data required to support Test Cases
- Design the test environment set-up
- Identify any required infrastructure and tools

Implementation and Execution

- Develop, implement and priorities
Test case
- Create the Test Scripts
- Create test data
- Write automated test scripts
- Check the environment – Verify
that the test environment has
been set up correctly

"Ideas are Easy. Implementation is hard"



Evaluating exit criteria and reporting

“Evaluation- Learning tool to Improve”

- Evaluating exit criteria is the activity where test execution is assessed against the defined objectives.
- Exit criteria should be set and evaluated for each test level
- Check test logs against the exit criteria specified in the test planning
- Assess if more tests are needed or if the exit criteria specified should be changed.
- Write a test summary report to stakeholders

Evaluating exit criteria and reporting

- How to measure exit criteria?
 - All the planned requirements must be met
 - All the high Priority bugs should be closed
 - All the test cases should be executed
 - If the scheduled time out is arrived
 - Test manager must sign off the release

*Note: All these parameters can be met by percentages
(not 100%)*

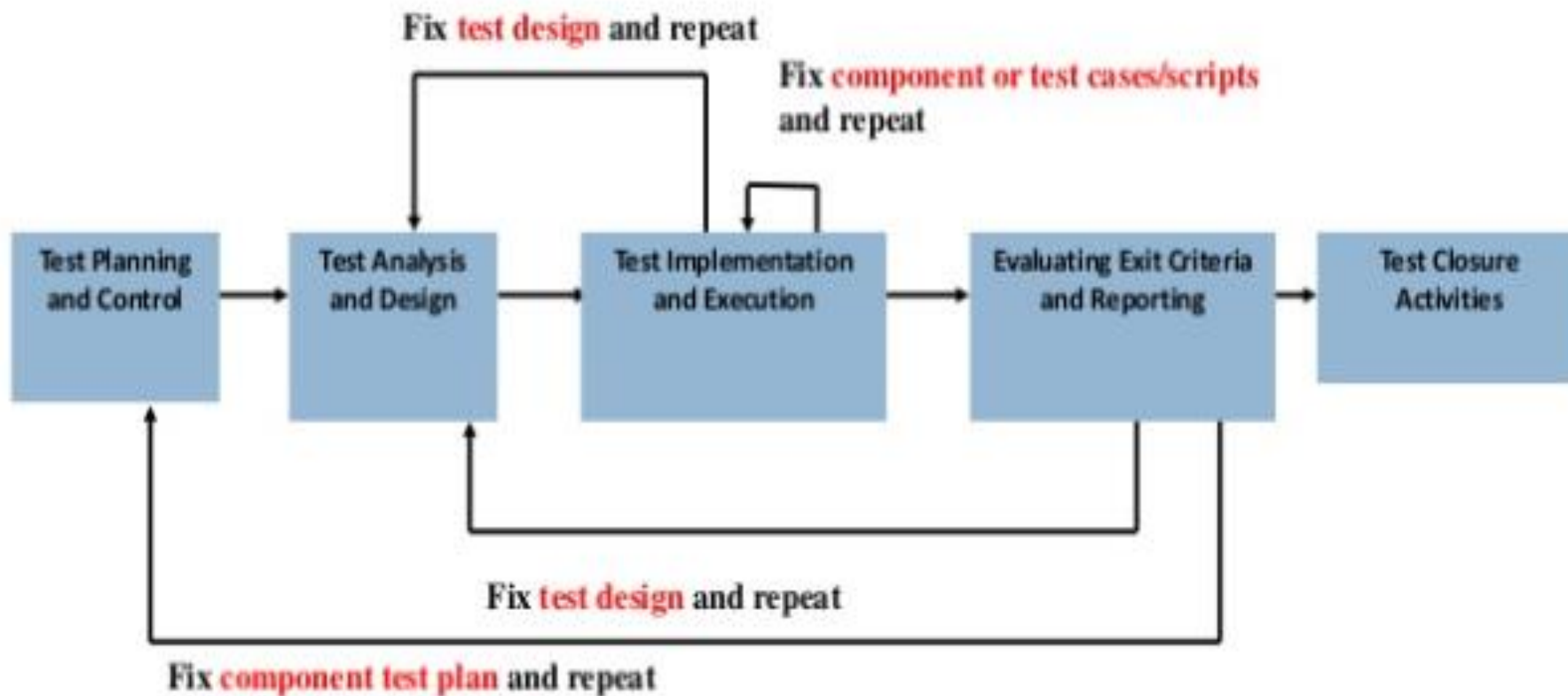
Test closure activities

“Closure – Way to start new beginning”

- Collect data from complete test activity
- Finalize and archive the test ware
- Test wares such as scripts, test environment etc.
- Evaluate how testing went and analyze lessons learned for future releases and projects



Fundamental Test Process





Chapter 5. Software Testing in the SDLC



Contents

- 1 The Role of Software Testing in SDLC
- 2 Definitions: Verification, Validation, QA, QC
- 3 Test Levels
- 4 Types of Testing

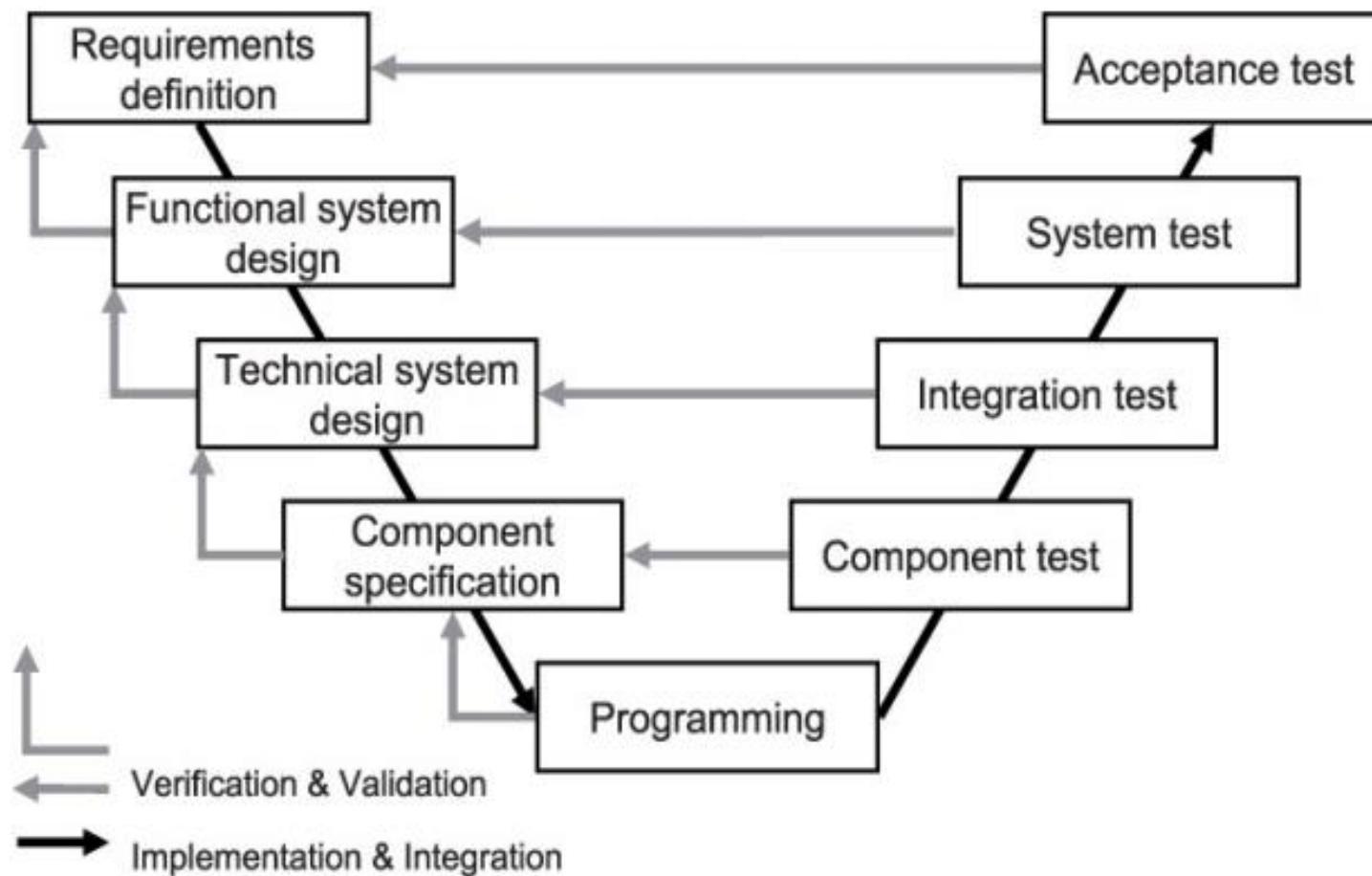
1. The Role of Software Testing in SDLC

- Software development models:
 - A development life cycle for a software product involves capturing the initial requirements from the customer, expanding on these to provide the detail required for code production, writing the code and testing the product, ready for release.
- There are two common models:
 - Sequential development models: Waterfall, V-model,...
 - Iterative and incremental development models: Scrum, Spiral, Agile, Kanban, Rational Unified Process,...

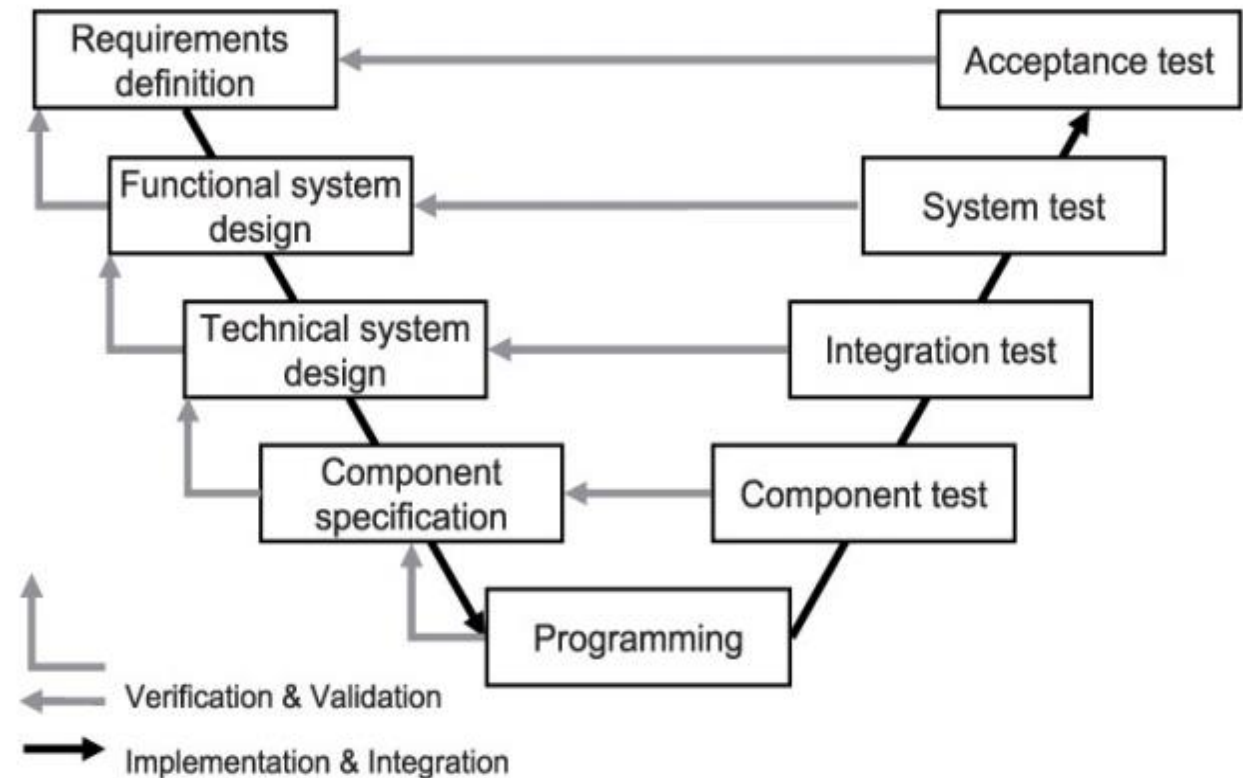


- The two branches of the V symbolize this.

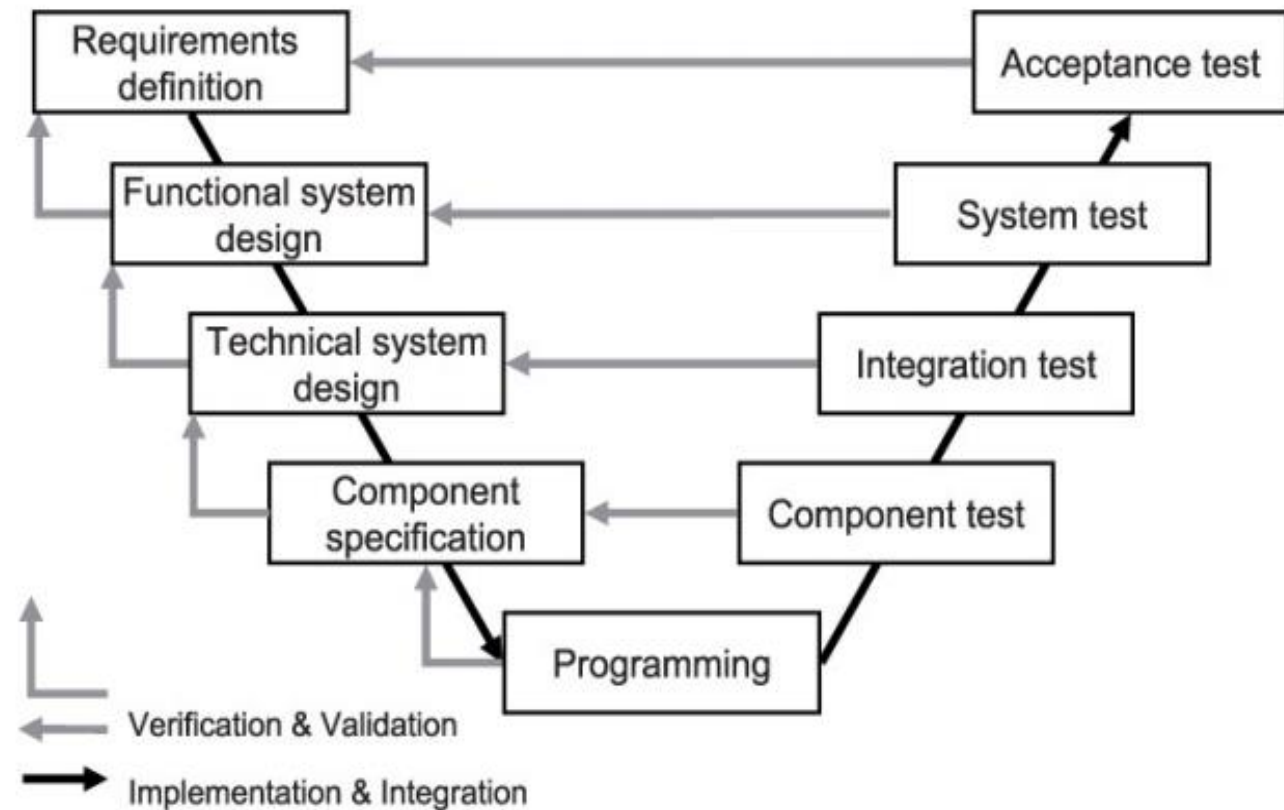
V-Model



- The left branch represents the development process:
 - Requirements definition
 - capturing of user needs
 - Functional specification
 - Definition of functions required to meet user needs
 - Technical system design
 - technical design of functions identified in the functional specification
 - Component specification
 - detailed design of each module or unit to be built to meet required functionality
 - Programming: use programming language to build the specified component (module, unit, class...)



- The right branch defines a test level for each specification and construction level
 - Component test:
 - Verifies whether each software, component correctly fulfills its specification.
 - Integration test
 - Checks if groups of components interact in the way that is specified by the technical system design.
 - System test
 - Verifies whether the system as a whole meets the specified requirements.
 - Acceptance test
 - Checks if the system meets the customer requirements, as specified in the contract and/or if the system meets user needs and expectations.



V-Model

- Some most important characteristics behind the V-model:
 - For every development activity, there is a corresponding testing activity.
 - Each test level has test objectives specific to that level.
 - Test analysis and design for a given test level begin during the corresponding development activity.
 - Tester should be involved in reviewing documents as soon as drafts are available in the development life cycle.
 - V-model illustrates the testing aspects of verification and validation.



Verification

Are we building
the product **right**?

To ensure that work products
meet their specified
requirements.



Validation

Are we building
the **right** product?

To ensure that the product
actually meets the user's needs,
and that the specifications
were correct in the first place.



2.2 Quality Assurance vs Quality Control

- **Quality Control:**

- Is a process that focuses on fulfilling the quality requested.
- Aims to identify and fix defects
- QC involves in full software testing life cycle
- QC activities are only a part of the total range of QA activities.

- **Quality Assurance:**

- It is a process that focuses on providing assurance that quality request will be achieved.
- Aims to prevent causes of detect and correct them early in the development process
- QA involves in full software development life cycle



3. Test Levels

- Test level (Test stage) is a group of test activities that are organized and managed together. A test level is linked to the responsibilities in a project.
- Test level consists of:
 - Component Testing
 - Integration Testing
 - System Testing
 - Acceptance Testing
 - Regression Testing

3.1 Component Testing

- Is also known as unit testing, module testing, program testing
- Is the testing of individual program units, such as a procedures, functions, methods, or classes, in *isolation*.
- The goal of Unit testing is to *ensure that the code written for the unit meets its specification*, prior to its integration with other units.
- Testing
 - Functionality
 - Non- functional characteristics

3.1 Component Testing

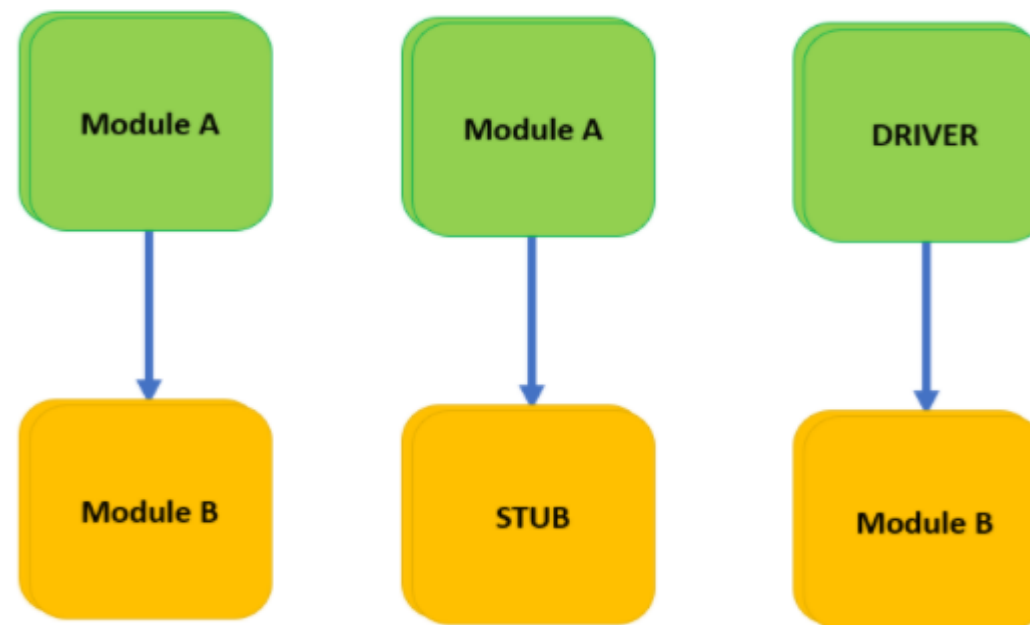
- Component testing is (usually) performed by the developer
- In test-driven development (TDD), which relates to Agile Software Development model, unit tests are prepared by developers *before code was written*.
- As unit testing is made in isolation from the rest of the system, there can be missing parts of software

=>Stubs and Drivers are used to replace the missing software and simulate the interface between the software components in a simple manner.



3.1 Component Testing

- A stub is called from the software component to be tested; a driver calls a component to be tested
- Stub and driver:



3.1 Component Testing

- Stub and driver:

Stubs	Drivers
Stubs are used in top down testing approach	Drivers are used in bottom up testing approach
Stubs are used when the major module is ready to test, but the sub modules are not ready yet	Drivers are used when the sub modules are ready, but the main module is not ready yet
Stubs are "called" programs	Drivers are "calling" programs

3.2 Integration testing

- Once the units have been written, the next stage is to put them together to create the system. This is called integration. It involves building something large from a number of smaller pieces.
- Integration testing *tests interfaces between components, interactions to different parts of a system* such as an operating system, file system and hardware or *interfaces between systems*.



3.2 Integration testing

- Levels of integration testing:
 - **Component integration testing**
 - Verifies the interactions between software components and is done after component testing.
 - is usually carried out by **developers**.
 - **System integration testing**
 - Verifies the interactions between different systems and may be done after system testing each individual system.
 - For example, a trading system in an investment bank will interact with the stock exchange to get the latest prices for its stocks and shares on the international market.
 - This type of integration testing is usually carried out by **testers**.



3.2 Integration testing

- There are three commonly integration strategies:
 - Big-bang integration
 - Incremental integration
 - Top-down integration
 - Bottom-up integration

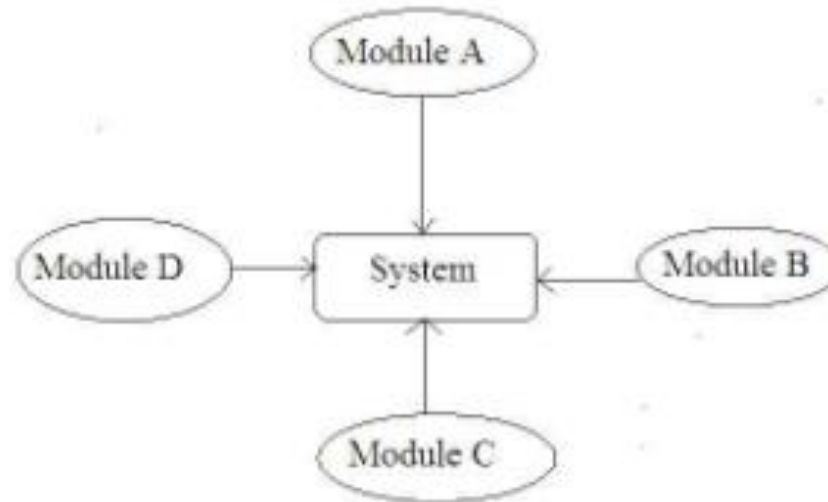


Big-Bang Integration

- In theory:
 - if we have already tested components why not just combine them all at once? Wouldn't this save time?
 - (based on false assumption of no faults)
- In practice:
 - takes longer to locate and fix faults
 - re-testing after fixes more extensive
 - end result? takes more time

- Big-bang integration:

- testing all components or modules are integrated simultaneously, after which everything is tested as a whole.
- Individual modules are not integrated until and unless all the modules are ready.



- Big-bang integration:

- Advantage: everything is finished before integration testing starts.
- Disadvantages:
 - in general, it is very time consuming
 - It is very difficult to trace the cause of failures because of this late integration.
 - The chances of having critical failures are more because of integrating all the components together at same time.
 - If any bug is found then it is very difficult to detach all the modules in order to find out the root cause of it.
 - There is high probability of occurrence of the critical bugs in the production environment

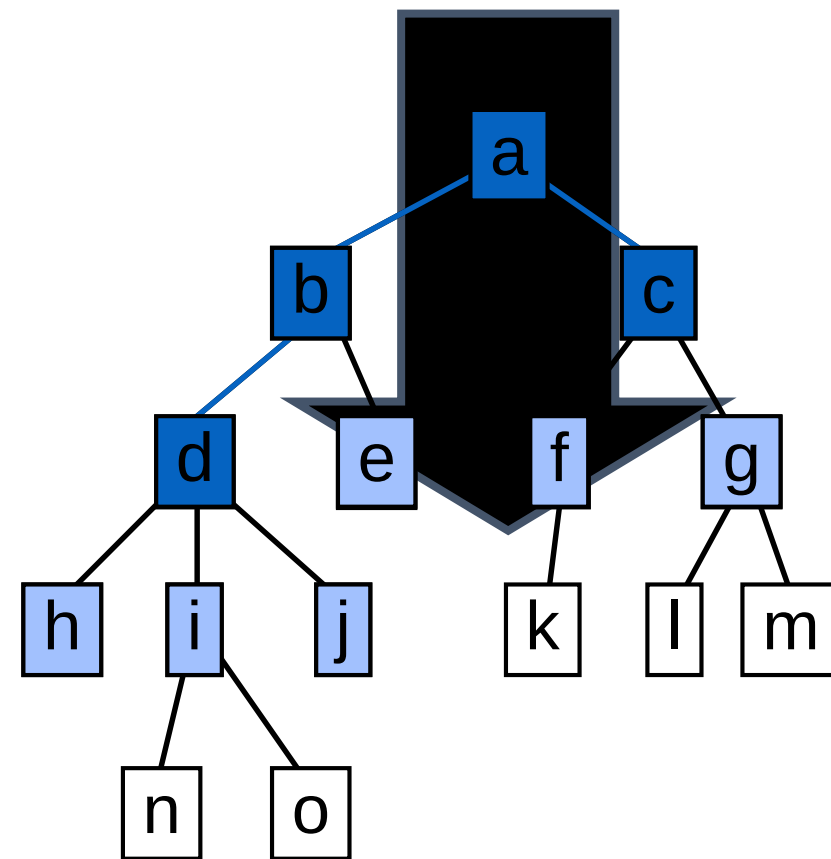


Incremental Integration

- Baseline 0: tested component
- Baseline 1: two components
- Baseline 2: three components, etc.
- Advantages:
 - easier fault location and fix
 - easier recovery from disaster / problems
 - interfaces should have been tested in component tests, but ..
 - add to tested baseline

Top-Down Integration

- Baselines:
 - baseline 0: component a
 - baseline 1: a + b
 - baseline 2: a + b + c
 - baseline 3: a + b + c + d
 - etc.
- Need to call to lower level components not yet integrated
- Stubs: simulate missing components



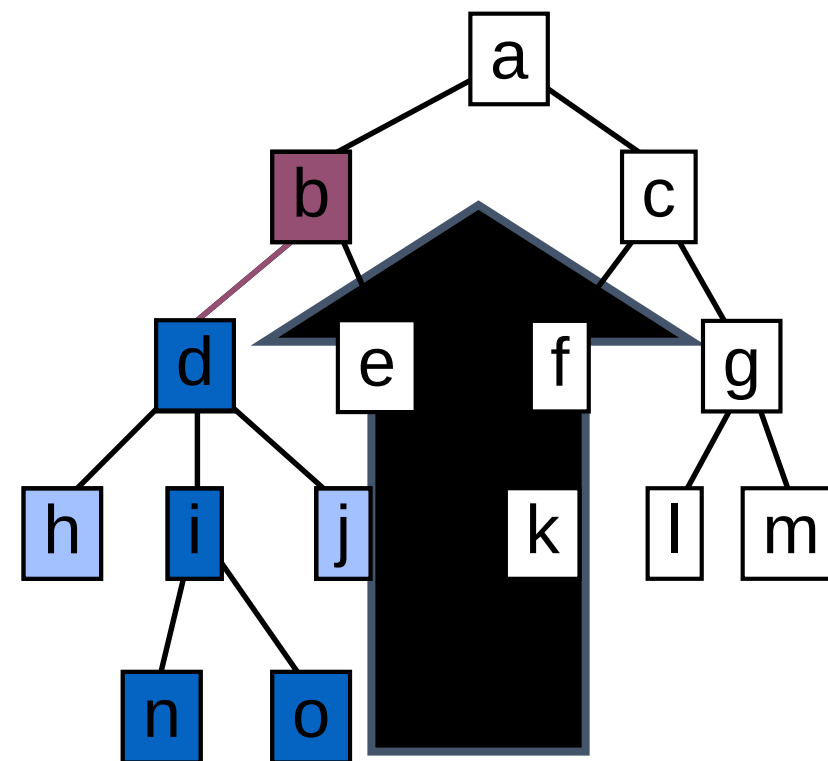
Pros & cons of top-down approach

- Advantages:
 - critical control structure tested first and most often
 - can demonstrate system early (show working menus)
- Disadvantages:
 - needs stubs
 - detail left until last
 - may be difficult to "see" detailed output (but should have been tested in component test)
 - may look more finished than it is



Bottom-up Integration

- Baselines:
 - baseline 0: component n
 - baseline 1: n + i
 - baseline 2: n + i + o
 - baseline 3: n + i + o + d
 - etc.
- Needs drivers to call the baseline configuration
- Also needs stubs for some baselines

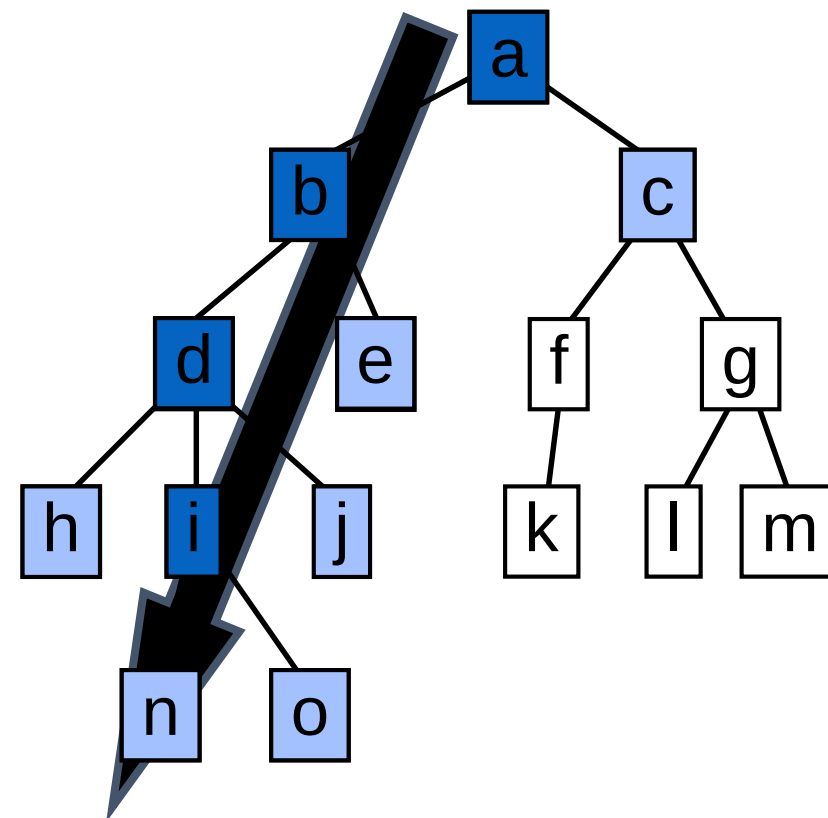


Pros & cons of bottom-up approach

- Advantages:
 - lowest levels tested first and most thoroughly (but should have been tested in unit testing)
 - good for testing interfaces to external environment (hardware, network)
 - visibility of detail
- Disadvantages
 - no working system until last baseline
 - needs both drivers and stubs
 - major control problems found last

Minimum Capability Integration (also called Functional)

- Baselines:
 - baseline 0: component a
 - baseline 1: a + b
 - baseline 2: a + b + d
 - baseline 3: a + b + d + i
 - etc.
- Needs stubs
- Shouldn't need drivers (if top-down)



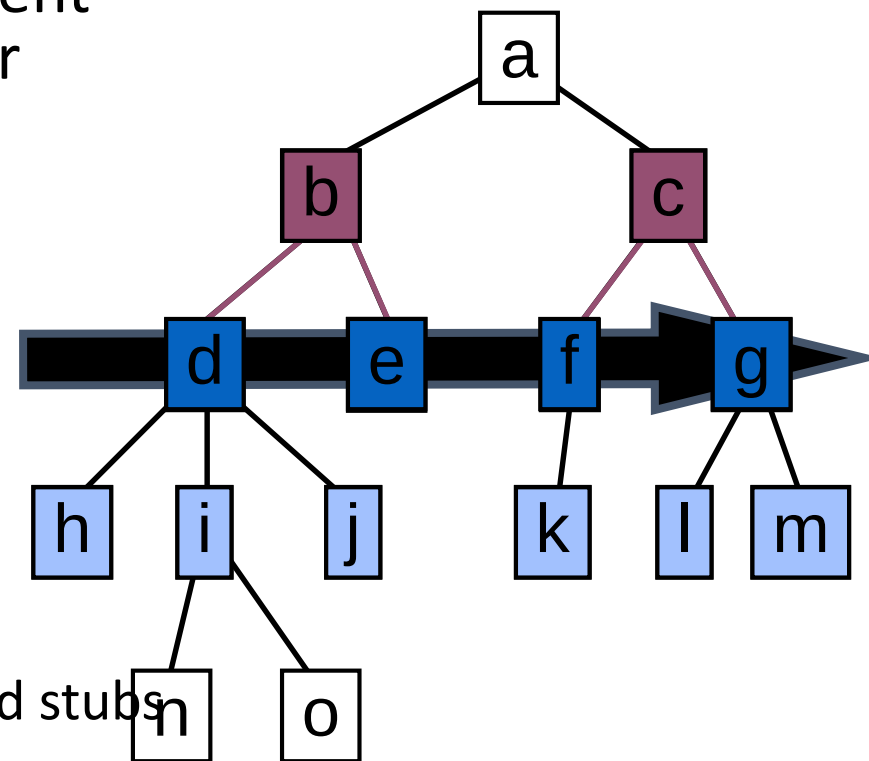
Pros & cons of Minimum Capability

- Advantages:
 - control level tested first and most often
 - visibility of detail
 - real working partial system earliest
- Disadvantages
 - needs stubs



Thread Integration (also called functional)

- order of processing some event determines integration order
- interrupt, user transaction
- minimum capability in time
- advantages:
 - critical processing first
 - early warning of performance problems
- disadvantages:
 - may need complex drivers and stubs



Integration Guidelines

- Minimise support software needed
- Integrate each component only once
- Each baseline should produce an easily verifiable result
- Integrate small numbers of components at once
 - one at a time for critical or fault-prone components
 - combine simple related components



Integration Planning

- Integration should be planned in the architectural design phase
- The integration order then determines the build order
 - components completed in time for their baseline
 - component development and integration testing can be done in parallel - saves time



3.3. System testing

- System testing is testing an integrated system to verify that it meets specified requirements.
- It may include tests based on risks and/or requirement specifications, business process, use cases, or other high level descriptions of system behavior, interactions with the operating systems, and system resources.
- System testing is carried out by specialists testers or independent testers.



3.3. System testing

- System testing:
 - non-functional tests include performance and reliability. Testers may also need to deal with incomplete or undocumented requirements.
 - functional requirements starts by using the most appropriate specification-based (black-box) techniques
- System testing requires a product environment



3.4 Acceptance testing

- Acceptance testing is software testing performed on software prior to its delivery.
- The goal of acceptance testing is not to find defects, but to provide the end users with confidence that the system will function according to their expectations and can be released.
- How much acceptance testing?
 - Depend on the product risk.



3.4 Acceptance testing

- There are four typical forms of acceptance testing:
 - Contract and regulation acceptance testing
 - User acceptance testing (UAT)
 - Operational acceptance testing (OAT)
 - Alpha and beta testing



3.4 Acceptance testing

- Alpha and beta testing are two stages of acceptance testing
 - Alpha testing:
 - Is testing by a potential users or an independent test team *at the developing organization's site* before the software product is released to external customers.
 - It is a form of internal acceptance testing.
 - Beta testing:
 - Is testing by a group of customers/potential users who use the product at their own locations and provide feedback *at the customer's site* , before the software product is released.
 - It is a form of external acceptance testing.



3.4 Acceptance testing

- Alpha and beta testing:
 - are usually performed on commercial off-the-shelf software (COTS). It is software that was developed for the mass market.
 - Done to get feedback from potential or existing users/customer before the software product is released to the market
 - Alpha testing is done before Beta testing



4. Test type

- Test Type is a group of test activities aimed at testing a component or system focused on a specific test objective.
- Test types:
 - Functional testing.
 - Non-functional testing.
 - Structural testing.
 - Testing related to changes (Retesting and Regression testing)

4.1 Functional testing

- Functional testing is testing of functions.
 - Objective: test each function of the software application, by providing appropriate input, verifying the output against the Functional requirements.

Testing what the system does

- Functional testing is also called **specification-based testing**

4.1 Functional testing

- The functionality of the application is usually described in the following documents:
 - Functional specifications
 - Requirements specifications
 - Business requirements
 - Use cases
 - User stories...
- ❖ Functional testing mainly involves black box testing and it is not concerned about the source code of the application

4.1 Functional testing

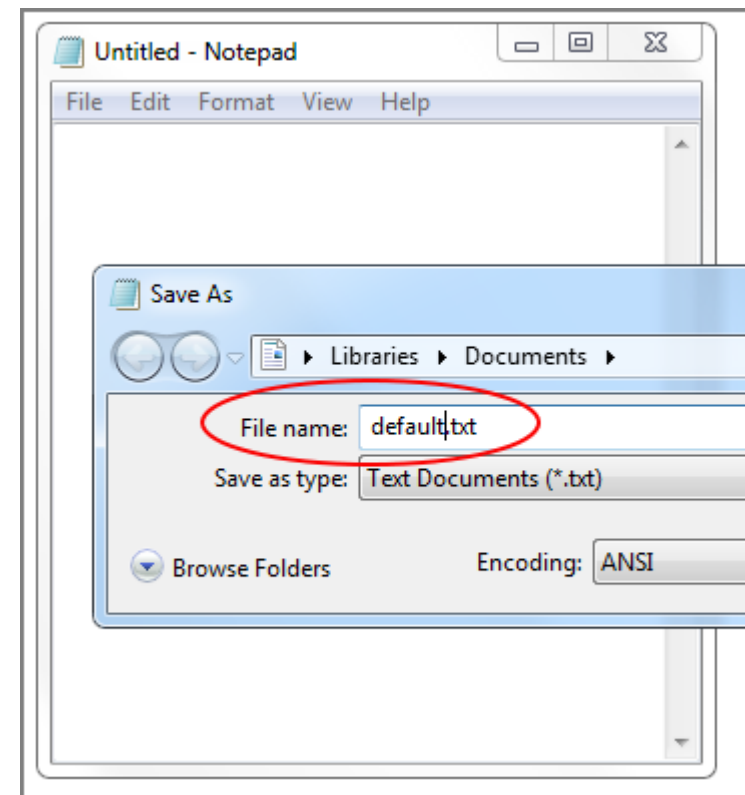
- Functional testing focuses on:
 - Security
 - Suitability
 - Accuracy
 - Interoperability (compatibility)
 - Compliance
- Example:
 - User can login the website successfully with valid account
 - User is able pay for the purchase in the e-store using Visa
 - Firewall can detect threats such as virus

4.1 Functional testing example

Task: Test Save feature of Notepad application.

Functional Testing Procedure: test different flows of Save functionality (Save new file, save updated file, test Save As, save to protected folder, save with incorrect name, re-write existed document, cancel saving, etc.)

Defect: While trying to save file using Save As command, still default file name can only be used. User cannot change the filename because the edit-box is disabled.



4.2 Non-Functional testing

- Non-Functional testing is testing of Non-functional software characteristics

Testing of how the system works

- Objective: a non-functional characteristics that do not relate to functionality of a software application.
- Non-functional characteristics are:
 - Reliability
 - Usability
 - Efficiency
 - Maintainability
 - Portability

4.2 Non-Functional testing

- Types of Non-Functional testing:
 1. Performance testing
 - Load testing
 - Stress testing
 - Endurance testing
 - Volume testing
 - Spike Testing
 2. Document Testing
 3. Installation Testing
 4. Reliability Testing
 5. Security Testing



4.2.1 Performance testing

- Performance testing is a type of performance testing to determine how efficiently a product handles a variety of events.
 - Ex: Measuring the response time of a website, or a specific element
- Tester needs to check product's speed, scalability and stability
 - Speed : Need to verify that application response is getting quick or not.
 - Scalability : Need to verify that application can handle maximum user load.
 - Stability : Need to verify that application is stable under varying load.

4.2.1 Performance testing: Example

Task: Server should respond in less than 2 sec when up to 100 users access it concurrently. Server should respond in less than 5 sec when up to 300 users access it concurrently.

Performance Testing Procedure: emulate different amount of requests to server in range (0; 300), for instance, measure time for 10, 50, 100, 240 and 290 concurrent users.

Defect: starting from 200 Concurrent requests respond time is 10-15 seconds.



4.2.2.1 Load testing

- Load testing is testing which involves evaluating the performance of the system under the *expected workload* to determine what load can be handled by the component or system.
- Expected workload: numbers of parallel users, numbers of transactions...
- Example:
 - Uploading/downloading big volume files
 - Ordering item in online store by a big amount of users simultaneously



4.2.2.1 Load testing: Example

Task: Server should allow up to 500 concurrent connections.

Load Testing Procedure: emulate different amount of requests to server close to pick value, for instance, measure time for 400, 450, 500 concurrent users.

Defect: Server returns "Request Time Out" starting from 490 concurrent requests.



4.2.2.2 Stress testing

- Stress testing: is a type of performance testing conducted to evaluate a system or component at or beyond the limits of its anticipated or specified work loads, or with reduced availability of resources such as access to memory or servers



4.2.2.2 Stress testing: Example

Task: Server should allow up to 500 concurrent connections.

Stress Testing Procedure: emulate amount of requests to server greater than pick value, for instance, check system behavior for 500, 510, and 550 concurrent users.

Defect: Server crashes starting from 500 concurrent requests and user's data is lost.
Data should not be lost even in stress situations. If possible, system crash also should be avoided.



4.2.2.3 Endurance testing

- Endurance testing is a testing of the software to check system performance under specific load conditions over an extended or longer amount of time.
- It is also known as Soak testing



4.2.2.3 Endurance testing: Example

- A system may behave exactly as expected when tested for 1 hour but when the same system is tested for 3 hours, problems such as memory leaks cause the system to fail or behave randomly.
- For an application like Income tax filing, the application is used continuously for a very long duration by different users. In this type of application, memory management is very critical.



4.2.2.4 Volume testing

- Volume testing refers to testing a software application or the product with a certain amount of data.
- E.g., if we want to volume test our application with a specific database size, we need to expand our database to that size and then test the application's performance on it.



Type of testing in which the system is tested
on large data volumes

4.2.2.5 Spike Testing

- It refers to test conducted by subjecting the system to a short burst of concurrent load to the system. This test might be essential while conducting performance tests on an auction site wherein a sudden load is expected.
- **Example** – For an e-commerce application running an advertisement campaign, the number of users can increase suddenly in a very short duration.

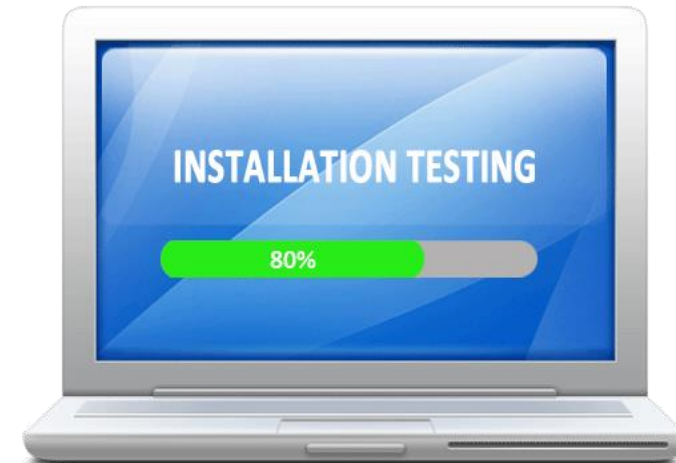


4.2.2. Document Testing

- Document Testing means verify the technical accuracy and readability of the user manuals, tutorials and the online help.
- Document testing:
 - a spelling and grammar checking in the documents using available tools
 - to manually reviewing the documentation to remove any ambiguity or inconsistency.
- Can start at the very beginning of the software process and hence save large amounts of money

4.2.2.3. Installation Testing

- Installation testing is performed to check if the *software has been correctly installed* with all the inherent features and that the product is working as per expectations.
- Also known as **implementation testing**
- It is done in the last phase of testing before the end user has his/her first interaction with the product.



4.2.2.4. Reliability Testing

- Reliability testing is performed to ensure that the software is reliable, it satisfies the purpose for which it is made, for a specified amount of time in a given environment and is capable of rendering a fault-free operation



4.2.2.5. Security Testing

- Security testing is a testing technique to determine if an information system protects data and maintains functionality as intended.
- Example:
 - A password should be in encrypted format
 - Application or System should not allow invalid users
 - Check cookies and session time for application

4.3 Structural testing

- The structural testing is the testing of the structure of the system or component.

Testing through assessment of coverage of a type of structure

- Structural testing is often referred to as ‘white box’ testing because in structural testing we are interested in what is happening ‘inside the system/application’.

4.3 Structural testing

- Structural testing can be used at all levels of testing
 - Developers use structural testing in component testing and component integration testing, especially where there is good tool support for **code coverage**.
- Coverage measurement tools assess the percentage of executable elements (e.g. statements or decision outcomes) that have been exercised (i.e. covered) by a test suite.



4.4 Testing related to changes

- Objective: changes related to defects that have been fixed and looking for unintended changes
- When a defect has been corrected, two types of tests should be executed:
 - Confirmation testing (retesting)
 - Regression testing



4.4 Testing related to changes

- **Confirmation testing(Re-testing)** is testing a defect again – after it has been fixed.
- When developers have fixed the defect, it is assigned for testers to re-test test cases with the same inputs, data and environment to **confirm** the defect was really fixed.



4.4 Testing related to changes

- **Regression testing** is a testing of a previously tested program following modification to ensure that defects have not been introduced or uncovered in **unchanged areas** of the software, as a result of the changes made.
- Changes can be made in the software or environment.
- New defect can appear when
 - Software was changed: adding/fixing functionality, refactoring...
 - Environment was changed

4.4 Testing related to changes

Regression Testing	Re-Testing
In Regression testing, you can include the test cases which passed earlier. We can say that check the functionality which was working earlier.	In Retesting, you can include the test cases which failed earlier. We can say that check the functionality which was failed in earlier build.
Regression testing can done by Automation testing	Retesting cant be done by Automation testcases
Defect verification is not the part of Regression Testing	Defect verification is the part of re-testing
Regression testing checks for unexpected side-effects	Re-testing makes sure that the original fault has been corrected
Regression testing is only done when there is any modification or changes become mandatory in an existing project	Re-testing executes a defect with the same data and the same environment with different inputs with a new build project

Regression testing = find new bugs.

Retesting = confirm bugs were fixed.

Regression testing locates bugs after updates or changes to the code or UI.

Retesting makes sure that a bug that was found and addressed by a developer was *actually* fixed

